

NAVSHIELD: AI-ENHANCED MARITIME MONITORING AND PREDICTIVE SAFETY ALERTS

CS680506-MAIN PROJECT REPORT

Submitted by

V MATHANKUMAR (95222203043)

in partial fulfilment for the award of the degree of



BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

**PSN COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)**

MELATHEDIYOOR, TIRUNELVELI 627 152

MARCH 2026

PSN COLLEGE OF ENGINEERING AND TECHNOLOGY (AUTONOMOUS)

BONAFIDE CERTIFICATE

Certified that this report, titled “ **NAVSHIELD : AI-ENHANCED MARITIME MONITORING AND PREDICTIVE SAFETY ALERTS**” is the bonafide work of **V MATHANKUMAR (95222203043)** , who carried out the work under my supervision.

Dr. M. VARGHEESE, M.TECH., Ph.D.,

Professor and Head,

Department of Computer Science and
Engineering ,

PSN College of Engineering and Technology,

Tirunelveli – 627152.

Mr. D. JOSEPH PUSHPARAJ, M.E.,

Professor and Head,

Department of Artificial Intelligence
with Machine Learning,

PSN College of Engineering and Technology,

Tirunelveli – 627152.

Submitted for Viva-Voce Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER



Vision and Mission of the Institute

Vision

- Emerge as a pioneer institute inculcating engineering education and skills, research, values and ethics.

Mission

- To achieve greater heights of excellence in technical knowledge and skill development through innovative teaching and learning practices.
- To develop the state of art infrastructure to meet the demands of technological revolution.
- To improve and foster research in all dimensions for betterment of society.
- To develop individual competencies to enhance innovation, employability and entrepreneurship among students.
- To instill higher standards of discipline among students, inculcating ethical and moral values for societal harmony and peace.

Vision and Mission of the Department

Vision

- To emerge as a preeminence program to produce quality Computer Science and Engineering graduates .

Mission

- To enhance professional and entrepreneurial skills through industry institute interaction to enable them in getting better placement
- To promote research and continuing education
- To train the students according to their discipline to meet dynamic needs of the society

ABSTRACT

Maritime security faces challenges from illegal crossings, smuggling, and environmental hazards, while fishermen are exposed to risks due to limited real-time information. This project proposes FBAS, an integrated border alert system leveraging LSTM neural networks for predictive border classification. The system combines weather monitoring via API, hazard alerts, and timely notifications to maritime authorities and stakeholders, enhancing surveillance, response, and maritime safety. By bridging gaps in traditional systems, FBAS provides comprehensive protection for maritime borders and supports safe maritime operations.

ACKNOWLEDGEMENT

Our first and foremost thanks to THE ALMIGHTY for presenting this opportunity to complete this project.

I thank our Chairman **Dr. P. SUYAMBU, Ph.D.**, Founder and Chairman, **PSN GROUP OF INSTITUTIONS** for permitting me to do this project work and for his kind patronage.

I thank our Vice Chairmen **ER.P.S. JEGANNATH** and **ER.P.S. JAYARAM** for permitting me to do this project work.

I extend our gratitude to **Dr.P. Selva Kumar, M.E., Ph.D.**, Executive Director, and **Dr. S.P. Umayal, M.E., Ph.D.**, Principal, for their valuable guidance and support

I express our special thanks to **Dr.M. Vargheese, M.Tech., Ph.D.**, Head of the Department, for his continuous encouragement and valuable suggestions.

It gives an immense pleasure to thank **Mr. D. JOSEPH PUSHPARAJ M.E.**, Head of the Department of Artificial Intelligence with Machine Learning, for his expert guidance and constant support throughout this project.

Our special thanks to our faculty Advisor **Mr .N. PONNITHISH, M.E**, Assistant Professor Department of Computer Science and Engineering for his profound and valuable insight and comments on my project.

We thank all of our staff member for their valuable cooperation, guidance and suggestion given to do this project work.

TABLE OF CONTENTS

S. NO	TITLE	PAGE NO
	ABSTRACT	iv
	LIST OF TABLES	vii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	1
	1.1. Overview	1
	1.2. Problem Statement	2
	1.3. Aim and Objective	3
	1.4. Scope of the Project	4
2	LITERATURE SURVEY	5
	2.1. IoT Assisted Fisherman Aid	5
	2.2. Arduino based Maritime Border Alert System for Fisherman	6
3	SYSTEM ANALYSIS	8
	3.1. Existing System	8
	3.1.1. Disadvantage	10
	3.2. Proposed System	11
	3.2.1. Advantages	12
4	SYSTEM CONFIGURATION	13
	4.1. Hardware Specification	13
	4.2. Software Specification	13
5	SOFTWARE DESCRIPTION	14
	5.1. Python	14
	5.2. MySQL	17
	5.3. WampServer	18
	5.4. Bootstrap 4	20

	5.5. Flask	20
6	SYSTEM DESIGN	22
	6.1. System Architecture	22
	6.2. Dataflow Diagram	23
	6.3. UML Diagram	26
	6.4. Database Design	27
7	SYSTEM IMPLEMENTATION	29
	7.1. System Description	31
	7.2. System Flow	32
8	SYSTEM TESTING	41
	8.1. Test Cases	43
	8.2. Test Report	45
9	CONCLUSION AND FUTURE ENHANCEMENT	47
	9.1. Conclusion	47
	9.2. Future Enhancement	48
10	APPENDIX	49
	10.1 Source Coding	49
	10.2 Screenshot	61
	10.3 Reference	70

LIST OF TABLES

TABLE NO.	FIGURES	PAGE NO
6.4.1.	User Table	27
6.4.2	Vessel Tracking Table	27
6.4.3	Weather Data Table	28
6.4.4	Alert Table	28

LIST OF FIGURES

FIGURE NO.	FIGURES	PAGE NO
1.1.	Coastal Place	1
6.1.	System Architecture	22
6.2.1	Data Flow Diagram Level 0	23
6.2.2.	Data Flow Diagram Level 1	24
6.2.3.	Data Flow Diagram Level 2	25
6.3.1.	Use Case Diagram	26
10.2.1	Screenshot 1	61
10.2.2	Screenshot 2	61
10.2.3	Screenshot 3	62
10.2.4	Screenshot 4	62
10.2.5	Screenshot 5	63
10.2.6	Screenshot 6	63
10.2.7	Screenshot 7	64
10.2.8	Screenshot 8	64
10.2.9	Screenshot 9	65
10.2.10	Screenshot 10	65
10.2.11	Screenshot 11	66
10.2.12	Screenshot 12	66
10.2.13	Screenshot 13	67
10.2.14	Screenshot 14	67
10.2.15	Screenshot 15	68
10.2.16	Screenshot 16	68
10.2.17	Screenshot 17	69

LIST OF ABBREVIATIONS

S.NO	ABBREVIATION	EXPANSION
1	AI	Artificial Intelligence
2	FBAS	Forecast-Based Alert System
3	LSTM	Long Short-Term Memory
4	API	Application Programming Interface
5	GPS	Global Positioning System
6	DB	Database
7	CNN	Convolutional Neural Network
8	GUI	Graphical User Interface
9	SMS	Short Message Service
10	IoT	Internet of Things

CHAPTER 1

INTRODUCTION

1.1. OVERVIEW

Fishing is the activity of trying to catch fish. Fish are often caught as wildlife from the natural environment (fresh water or marine), but may also be caught from stocked bodies of water such as ponds, canals, park wetlands and reservoirs. Fishing techniques include hand-gathering, spearing, netting, angling, shooting and trapping, as well as more destructive and often illegal techniques such as electrocution, blasting and poisoning. The term fishing broadly includes catching aquatic animals other than fish, such as crustaceans (shrimp/lobsters/crabs), shellfish, cephalopods (octopus/squid) and echinoderms (starfish/sea urchins). The term is not normally applied to harvesting fish raised in controlled cultivations (fish farming).



Fig: 1.1. Coastal Place

Fishing has been an important part of human culture since hunter-gatherer times. It is one of the few food production activities that has persisted from prehistory into the modern age, surviving both the Neolithic Revolution and successive Industrial Revolutions. In addition to being caught to be eaten for food, fish are caught as recreational pastimes. India has a coastline of 7,517 km, of which the mainland accounts for 5,422 km. The Lakshadweep coast extends for 132 km

and the Andaman and Nicobar Islands have a coastline of 1,962 km. The Indian coastline is distributed among nine coastal states and four UTs, and almost the entire coast of India falls within the tropics. The nine coastal states are Gujarat, Maharashtra, Goa, Karnataka, Kerala, Tamil Nadu, Andhra Pradesh, Odisha and West Bengal. India also has large coastal wetlands, which cover an area of over 41,401 km², which is 27.13% of the total area covered by wetlands in India. India's inland wetlands, on the other hand, cover 1,05,649 km².

1.2. PROBLEM STATEMENT

The Fisherman Border Surveillance System is a pivotal element of maritime security strategies, dedicated to monitoring and fortifying maritime borders, particularly in regions susceptible to illicit activities and security risks. It encompasses a multifaceted array of technologies, protocols, and methodologies aimed at detecting, deterring, and responding to unauthorized border crossings by fishing vessels. It was relied on a combination of manual surveillance techniques and technological aids. Manual surveillance entails the deployment of coast guard personnel, stationed at lookout points or aboard patrol vessels, to visually scan the maritime domain for suspicious activities. These personnel rely on their training and experience to discern potential threats and breaches of maritime borders. Additionally, maritime patrols are conducted to enforce maritime laws, intercept suspicious vessels, and prevent illegal activities such as smuggling and human trafficking. Technological aids play a pivotal role in bolstering manual surveillance efforts and extending the reach of border monitoring capabilities. Radar installations positioned along coastlines offer continuous coverage and detection of vessel movements, enabling authorities to monitor maritime traffic in real-time. The Automatic Identification System (AIS) technology facilitates the tracking and identification of vessels by transmitting vessel-specific data, including identity,

position, course, and speed. This data is relayed to shore-based stations or satellite systems, empowering authorities to maintain situational awareness and identify vessels operating in restricted areas or exhibiting suspicious behavior. However, despite the efficacy of traditional surveillance methods and technological enhancements, Fisherman Border Surveillance Systems encounter several challenges and constraints. These include limited coverage due to geographical terrain and resource availability, dependence on human vigilance which is susceptible to errors and fatigue, integration complexities of disparate data sources, and scalability issues coupled with significant operational costs. To surmount these challenges and fortify the effectiveness of Fisherman Border Surveillance Systems, there is a burgeoning focus on harnessing advanced technologies such as artificial intelligence (AI), deep learning, unmanned aerial vehicles (UAVs), and sensor networks. These technologies offer the promise of automating detection processes, analyzing vast datasets in real-time, and furnishing early warnings of potential border breaches.

1.3. AIM AND OBJECTIVE

Aim

The aim of the project is to enhance maritime security along the Tamil Nadu to Sri Lanka border through the development and implementation of an integrated system that leverages data-driven approaches and predictive analytics to proactively monitor, detect, and respond to potential border breaches by fishing vessels.

Objectives

- To develop a user friendly web application for maritime security operations.
- To integrate deep learning for predictive border classification.
- To collect and preprocess diverse datasets for model training.
- To implement a real-time alerting system for immediate notifications.

- To integrate real-time weather alert functionality into the system.
- To incorporate hazard alert systems for enhanced safety measures.
- To ensure secure user authentication and access control.
- To enable continuous monitoring and analysis of maritime activity.
- To provide user training and support for effective system utilization.
- To establish an evaluation framework for performance assessment.
- To design for scalability and sustainability.

1.4. SCOPE OF THE PROJECT

The detailed scope of the project encompasses several key aspects:

- **Coast Guard Stations Web App Development:** The primary focus is on developing a web application using Python and Flask for the backend, MySQL for database management, and Bootstrap for the frontend. The app serves as a centralized platform for coast guard personnel to manage border security operations efficiently.
- **Integration of Machine Learning Models:** The project involves integrating machine learning capabilities using TensorFlow and Keras for predictive analytics. This includes developing models like BorderNet for classifying borders as safe or breached, enhancing the app's ability to predict and respond to potential threats.
- **Deployment of BorderAlert Devices:** Integrating BorderAlert GPS devices on fishing vessels enables continuous location tracking. This feature ensures that real-time vessel positions and trajectories are available to coast guard authorities, enhancing situational awareness and response capabilities.

CHAPTER 2

LITERATURE SURVEY

2.1. IoT Assisted Fisherman Aid to Detect Borders and Alert System by using Intelligent GPS Technology

Author: A P. Ramesh; R. Nithya Paranthaman;

Year2024

Doi: 10.1109/ICSES60034.2023.10465484

Problem

The safety of fishermen faces significant challenges due to the lack of effective monitoring and communication systems. Instances of fishermen entering restricted areas or encountering emergencies at sea highlight the urgent need for innovative solutions. Traditional methods fall short in providing timely interventions, necessitating the integration of advanced technologies such as Global Positioning System (GPS) and Wireless Sensor Networks (WSN) to enhance safety measures.

Objective

This project aims to improve the safety of fishermen by utilizing GPS and WSN technologies to enable real-time tracking, monitoring, and communication. Specifically, the objectives include developing a robust GPS-based tracking system for precise location monitoring, implementing a WSN for seamless communication between fishermen and a centralized system, creating an alert mechanism to warn fishermen approaching restricted areas, and designing an automated response system to mitigate risks and ensure prompt actions during emergencies.

Methodology

The methodology involves a comprehensive approach comprising research, development, testing, and deployment phases. Initially, extensive research is

conducted to understand the requirements and challenges faced by fishermen and to identify suitable technologies. Customized software and hardware solutions are developed based on the research findings to meet the specific needs of the fishing industry. Prototypes of the GPS tracking system, WSN, and alert mechanisms are then built and subjected to rigorous testing in simulated and real-world environments to ensure reliability and effectiveness. Following successful testing, the components are integrated into a cohesive system and deployed across a pilot group of fishing vessels

Finding

The implementation and testing of the integrated system demonstrated significant improvements in the safety and security of fishermen. Real-time tracking and communication capabilities provided by GPS and WSN technologies enabled proactive monitoring and rapid response to emergencies. The alert mechanisms effectively alerted fishermen when approaching restricted areas, reducing the risk of inadvertent violations. Additionally, the automated response system proved instrumental in mitigating risks and preventing accidents by initiating timely actions. Overall, the project successfully addressed the critical need for enhanced safety measures in the fishing industry, contributing to the well-being of fishermen and the sustainability of fishing operations.

2.2. Arduino based Maritime Border Alert System for Fisherman

Author: Lijo Jacob Varghese; A. Sakthivel

Year2022

Doi: 10.1109/ICICT54344.2022.9850676

Problem

Fishermen face precarious livelihoods, where accidental border crossings are treated as severe offenses, exacerbating their vulnerability. The situation escalates

tragically when impoverished fishermen face violence, including being shot at and having their boats confiscated. At the heart of this issue lies the difficulty in identifying maritime borders between countries, leading to inadvertent violations and ensuing consequences.

Objective

The primary aim is to develop a Maritime Border Alert System (MBAS) that aids fishermen by providing timely information about national borders, thereby safeguarding them and their vessels. This system intends to mitigate the risks associated with accidental border crossings and the subsequent punitive actions. To achieve this, technologies such as the Global Navigation Satellite System (GNSS), Universal Mobile Telecommunications Network (UMTS), and ultrasonic sensors are proposed to be integrated into a comprehensive framework

Methodology

The proposed framework entails the development of a low-cost Maritime Boundary Crossing Warning System tailored specifically for fishermen. The GNSS will provide accurate location data, enabling precise border identification. UMTS will facilitate communication between the system and fishermen, delivering timely alerts and guidance. Ultrasonic sensors will complement the system by detecting nearby vessels and potential hazards.

Finding

The implementation of the Maritime Border Alert System is anticipated to significantly enhance the safety and security of fishermen operating in border regions. By providing real-time information about maritime borders, the system will empower fishermen to navigate safely and avoid inadvertent violations. Moreover, the low-cost nature of the framework ensures accessibility to even the most economically disadvantaged fishermen, thereby promoting inclusivity and equity in maritime operations.

CHAPTER 3

SYSTEM ANALYSIS

3.1. EXISTING SYSTEM

The existing system of fisherman border surveillance relies on conventional methods and technologies, which differ significantly from modern, technology-driven approaches. Here's an outline of the traditional system:

Visual Surveillance

Historically, fisherman border surveillance predominantly involved visual monitoring by coast guard personnel stationed at lookout points along the coastline. This method relies on human observation using binoculars or telescopes to detect any suspicious activity or unauthorized border crossings by fishing vessels.

Manual Patrols

Coast guard patrols using boats or vessels are another traditional method of fisherman border surveillance. These patrols involve physically patrolling the maritime borders to deter and intercept any unauthorized vessels attempting to cross the borders.

Radio Communication

Communication between coast guard stations and fishing vessels traditionally occurs via marine VHF radio. Fishermen often report their positions and activities to coast guard authorities using radio channels designated for maritime communication.

Radar and AIS Integration

Radar systems and Automatic Identification System (AIS) technology are integrated into the existing surveillance infrastructure to track vessel movements in real-time. Radar systems detect vessels within the surveillance area, while AIS transponders on fishing vessels transmit vessel identification and position data, enabling authorities to monitor maritime traffic effectively.

GPS Tracking

Fishing vessels are equipped with GPS tracking devices that provide precise location data, allowing coast guard authorities to monitor vessel movements and ensure compliance with designated fishing zones and maritime borders. GPS tracking enhances situational awareness and enables timely response to border breaches or emergencies.

Remote Sensing Technologies

Satellite imagery and remote sensing technologies are utilized to monitor large maritime areas and detect suspicious activities, such as illegal fishing or unauthorized border crossings. High-resolution satellite imagery provides detailed insights into maritime activities, enabling authorities to detect and deter illicit activities effectively.

Existing Algorithms

The existing algorithms used in fisherman border alert systems encompass a variety of methods aimed at detecting and responding to potential security threats and breaches along maritime borders. Here are some of the key algorithms commonly employed in such systems:

Pattern Recognition Algorithms

Pattern recognition algorithms analyze historical data and surveillance information to identify patterns indicative of suspicious or unauthorized activities, such as abnormal vessel movements or deviations from typical fishing routes. Machine learning techniques, including clustering, classification, and anomaly detection algorithms, are often used to recognize patterns and anomalies in vessel behavior

Support Vector Machines (SVM)

SVM is a supervised learning algorithm used for classification tasks in fisherman border alert systems. SVM seeks to find the optimal hyperplane that separates different classes of maritime activities, such as normal fishing behavior and suspicious vessel movements, based on observed features and attributes.

K-Nearest Neighbors (KNN)

KNN is a non-parametric algorithm used for classification and anomaly detection in fisherman border alert systems. KNN classifies fishing vessel behavior based on the similarity of their feature vectors to those of neighboring instances in the dataset. KNN is particularly useful for detecting outliers and anomalies in maritime surveillance data.

3.1.1. DISADVANTAGES

- Lack real-time monitoring capabilities.
- Reliance on manual surveillance methods.
- Limited scalability for handling large datasets.
- Inability to predict potential security threats proactively.
- Limited effectiveness in detecting complex security threats.
- Lack of user-friendly interfaces for coast guard personnel and fishermen.
- Inadequate alerting mechanisms for timely response to breaches.
- Susceptible to bias and skewed predictions if trained on biased data.
- Require large amounts of labeled data for training, which may be costly and time-consuming to acquire.
- Limited interpretability, making it challenging to explain model predictions to stakeholders.

3.2. PROPOSED SYSTEM

The proposed system aims to create a comprehensive maritime security solution that leverages technology to improve border surveillance, communication, and response capabilities, ultimately enhancing safety and security in coastal regions. The proposed system of the project utilizes deep learning methodologies, particularly LSTM neural networks, to develop predictive models for border classification and maritime security. By harnessing the power of deep learning, the system can analyze complex patterns and temporal dependencies within maritime data, enabling accurate predictions of border breaches and potential security threats.

Coast Guard Stations Web App

The centerpiece of the system, this web application provides a user-friendly interface for coast guard personnel to manage border security operations. It includes features for real-time border surveillance, communication with authorities, and coordination with fishermen.

BorderNet Model

The BorderNet Model is a deep learning system that employs LSTM neural networks to classify maritime borders as either safe or breached. It analyzes various data, including vessel trajectories and environmental conditions, to predict border status in real-time. Integrated into the Coast Guard Stations Web App, it enhances maritime security by providing predictive analytics for proactive border surveillance.

Alert System

The alert system component of the project, designed to provide timely notifications to fishermen and maritime authorities about potential risks and hazards in maritime environments.

3.2.1. ADVANTAGES

- Real-time border monitoring enhances security.
- Predictive analytics for proactive border breach prevention.
- User-friendly interface for easy access by coast guard and fishermen.
- Timely alerts for breaches, weather, and hazards prompt action.
- Fishermen safety through hazard alerts.
- Data integration for comprehensive analysis.
- Customizable alert preferences for user needs.
- Automated monitoring reduces operational costs.
- Scalable for future expansion and integration.
- Informed decision-making via real-time insights.

CHAPTER 4

SYSTEM CONFIGURATION

4.1. HARDWARE SPECIFICATIONS

- **Processor:** Multi-core CPU for handling concurrent requests and computations.
- **RAM:** 16GB RAM to accommodate data processing and model training tasks.
- **Storage:** 256 SSD (Solid State Drive) storage with ample space for storing datasets, models, and system files.

4.2. SOFTWARE SPECIFICATIONS

- **Operating System:** Windows 10 or 11 (for Windows-specific development)
- **Programming Language:** Python (version 3.6 or higher)
- **Neural Network Framework:** TensorFlow
- **Data Processing Libraries:** Pandas, NumPy, Scikit Learn
- **Visualization Libraries:** Matplotlib, Seaborn
- **Web Framework:** Flask for implementing a web-based user interface,
- **Database Integration:** MySQL for storing and retrieving relevant data.
- **Integrated Development Environment (IDE):** IDLE
- **Web Technologies:** HTML, CSS, and JavaScript

CHAPTER 5

SOFTWARE DESCRIPTION

5.1. PYTHON 3.8

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum during 1985-1990. Like Perl, Python source code is also available under the GNU General Public License (GPL). This tutorial gives enough understanding on Python programming language.



Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages. Python is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain.

Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

Tensor Flow

TensorFlow is an end-to-end open-source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and gives developers the ability to easily build and deploy ML-powered applications.



TensorFlow provides a collection of workflows with intuitive, high-level APIs for both beginners and experts to create machine learning models in numerous languages. Developers have the option to deploy models on a number of platforms such as on servers, in the cloud, on mobile and edge devices, in browsers, and on many other JavaScript platforms. This enables developers to go from model building and training to deployment much more easily.

Keras

Keras is a deep learning API written in Python, running on top of the machine learning platform TensorFlow. It was developed with a focus on enabling fast experimentation. Keras is an API designed for human beings, not machines. Keras follows best practices for *reducing cognitive load*: it offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear & actionable error messages. Keras also gives the highest priority to crafting great documentation and developer guides.



Simple. Flexible. Powerful.

Keras 3 is a multi-backend deep learning framework, with support for JAX, TensorFlow, and PyTorch. Effortlessly build and train models for computer vision, natural language processing, audio processing, timeseries forecasting, recommender systems, etc.

Pandas

pandas are a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language. pandas are a Python package that provides fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.



Pandas is mainly used for data analysis and associated manipulation of tabular data in Data frames. Pandas allows importing data from various file formats such as comma-separated values, JSON, Parquet, SQL database tables or queries, and Microsoft Excel. Pandas allows various data manipulation operations such as merging, reshaping, selecting, as well as data cleaning, and data wrangling features. The development of pandas introduced into Python many comparable features of working with Data frames that were established in the R programming language. The panda's library is built upon another library NumPy, which is oriented to efficiently working with arrays instead of the features of working on Data frames.

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible.



Matplotlib is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-oriented API for embedding plots into applications using general-purpose GUI toolkits like Tkinter, wxPython, Qt, or GTK.

Scikit Learn

scikit-learn is a Python module for machine learning built on top of SciPy and is distributed under the 3-Clause BSD license.



Scikit-learn (formerly scikits. learn and also known as sklearn) is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support-vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

5.2. MYSQL

MySQL is a relational database management system based on the Structured Query Language, which is the popular language for accessing and managing the records in the database. MySQL is open-source and free software under the GNU

license. It is supported by Oracle Company. MySQL database that provides for how to manage database and to manipulate data with the help of various SQL queries. These queries are: insert records, update records, delete records, select records, create tables, drop tables, etc. There are also given MySQL interview questions to help you better understand the MySQL database.



MySQL is currently the most popular database management system software used for managing the relational database. It is open-source database software, which is supported by Oracle Company. It is fast, scalable, and easy to use database management system in comparison with Microsoft SQL Server and Oracle Database. It is commonly used in conjunction with PHP scripts for creating powerful and dynamic server-side or web-based enterprise applications. It is developed, marketed, and supported by MySQL AB, a Swedish company, and written in C programming language and C++ programming language. The official pronunciation of MySQL is not the My Sequel; it is My Ess Que Ell.

5.3. WAMP SERVER

WAMP Server is a reliable web development software program that lets you create web apps with MYSQL database and PHP Apache2. With an intuitive interface, the application features numerous functionalities and makes it the preferred choice of developers from around the world. The software is free to use and doesn't require a payment or subscription. Moreover, the program installs on the system automatically, so you can fine tune the server without making any changes to the 'Setting' files.



WAMP Server is a reliable web development software program that lets you create web apps with MySQL database and PHP Apache2. With an intuitive interface, the application features numerous functionalities and makes it the preferred choice of developers from around the world. WampServer also has a “TrayIcon” allowing you to manage and configure your servers simply, without touching the configuration files.

WAMP Server Features

- Apache Webserver
- MySQL DB Server
- MariaDB Server
- PHP Scripting language installed
- WAMP Server Tools
- PhpMyAdmin to manage DBs
- Manage Apache and MySQL services
- Switch to online / offline mode (accessible to all or limited to localhost)
- Install and change version of Apache, MySQL and PHP
- Manage the configuration parameters of your servers
- Access your logs
- Access configuration fi

5.4. BOOTSTRAP 4

Bootstrap is a powerful front-end framework for faster and easier web development. Bootstrap is a free and open-source web development framework. It's designed to ease the web development process of responsive, mobile-first websites by providing a collection of syntax for template designs. In other words, Bootstrap helps web developers build websites faster as they don't need to worry about basic commands and functions. It consists of HTML, CSS, and JS-based scripts for various web design-related functions and components.



It solves many problems which we had once, one of which is the cross-browser compatibility issue. Nowadays, the websites are perfect for all the browsers (IE, Firefox, and Chrome) and for all sizes of screens (Desktop, Tablets, Phablets, and Phones). All thanks to Bootstrap developers -Mark Otto and Jacob Thornton of Twitter, though it was later declared to be an open-source project. Bootstrap's primary objective is to create responsive, mobile-first websites. It ensures all interface elements of a website work optimally on all screen sizes. Bootstrap is available in two variants – precompiled and based on a source code version.

5.5. FLASK

Flask is a web framework. This means flask provides you with tools, libraries and technologies that allow you to build a web application. This web application can be some web pages, a blog, a wiki or go as big as a web-based calendar application or a commercial website.



Flask is often referred to as a micro framework. It aims to keep the core of an application simple yet extensible. Flask does not have built-in abstraction layer for database handling, nor does it have formed a validation support. Instead, Flask supports the extensions to add such functionality to the application. Although Flask is rather young compared to most Python frameworks, it holds a great promise and has already gained popularity among Python web developers. Let's take a closer look into Flask, so-called "micro" framework for Python. Flask is part of the categories of the micro-framework. Micro-framework is normally framework with little to no dependencies to external libraries. This has pros and cons. Pros would be that the framework is light, there are little dependency to update and watch for security bugs, cons is that some time you will have to do more work by yourself or increase yourself the list of dependencies by adding plugins.

CHAPTER 6

SYSTEM DESIGN

6.1. SYSTEM ARCHITECTURE

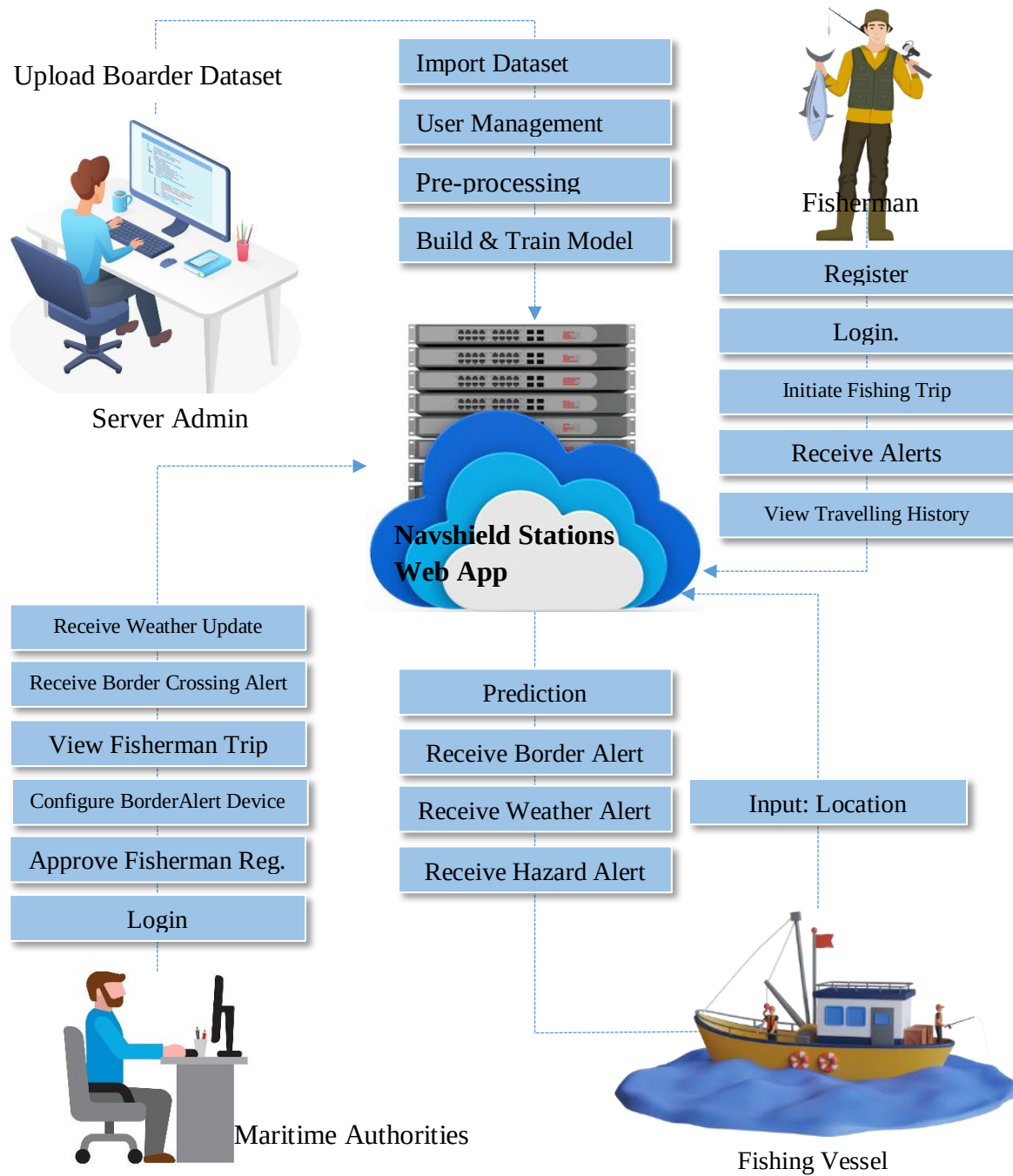


Fig. 6.1. System Architecture

6.2. DATA FLOW DIAGRAM

6.2.1. LEVEL 0

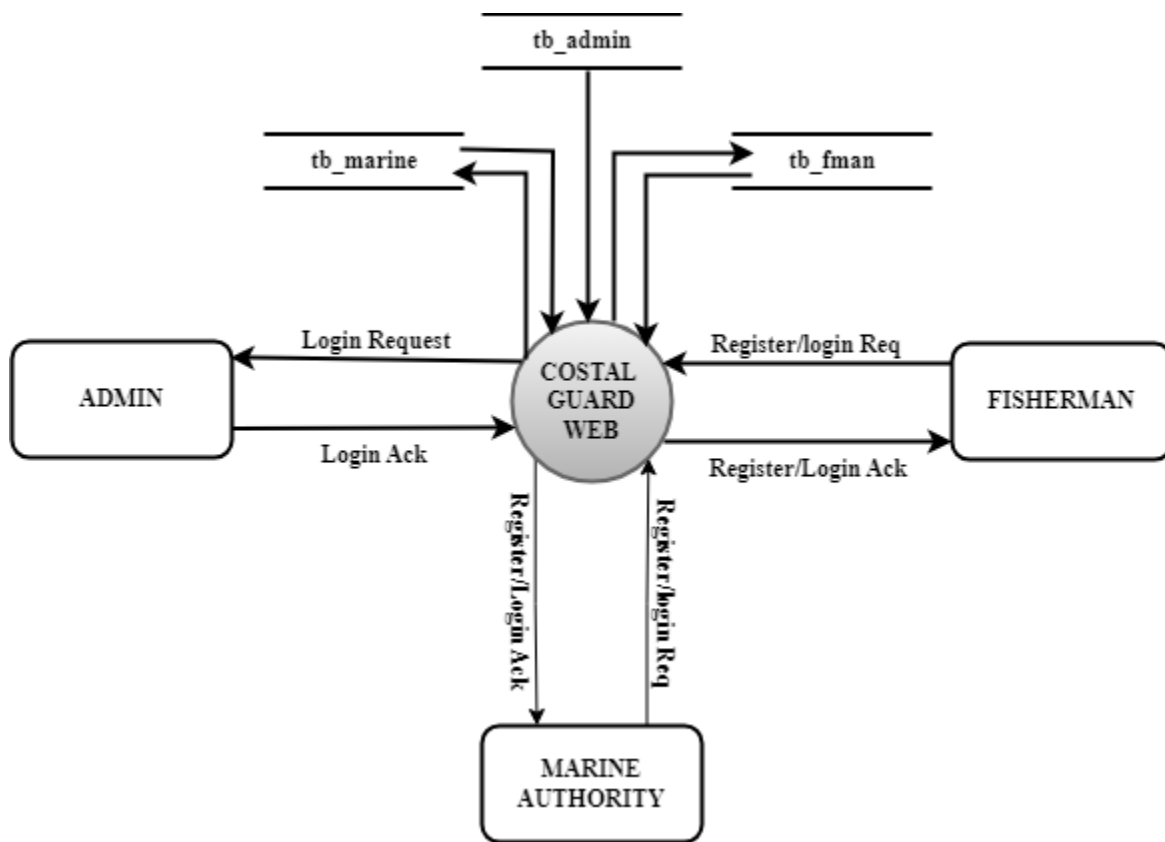


Fig. 6.2.1. Data Flow Diagram Level 0

6.2.2. 1-Level DFD

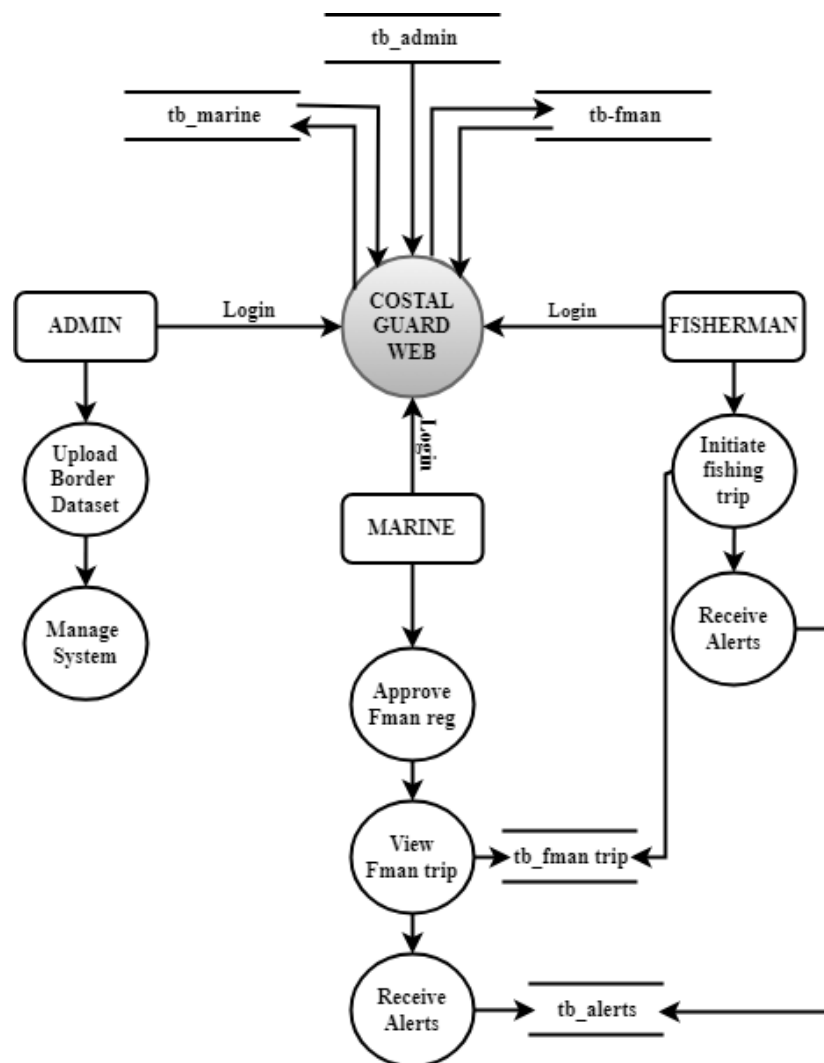


Fig. 6.2.2. Dataflow Diagram Level 1

6.2.3. 2-Level DFD

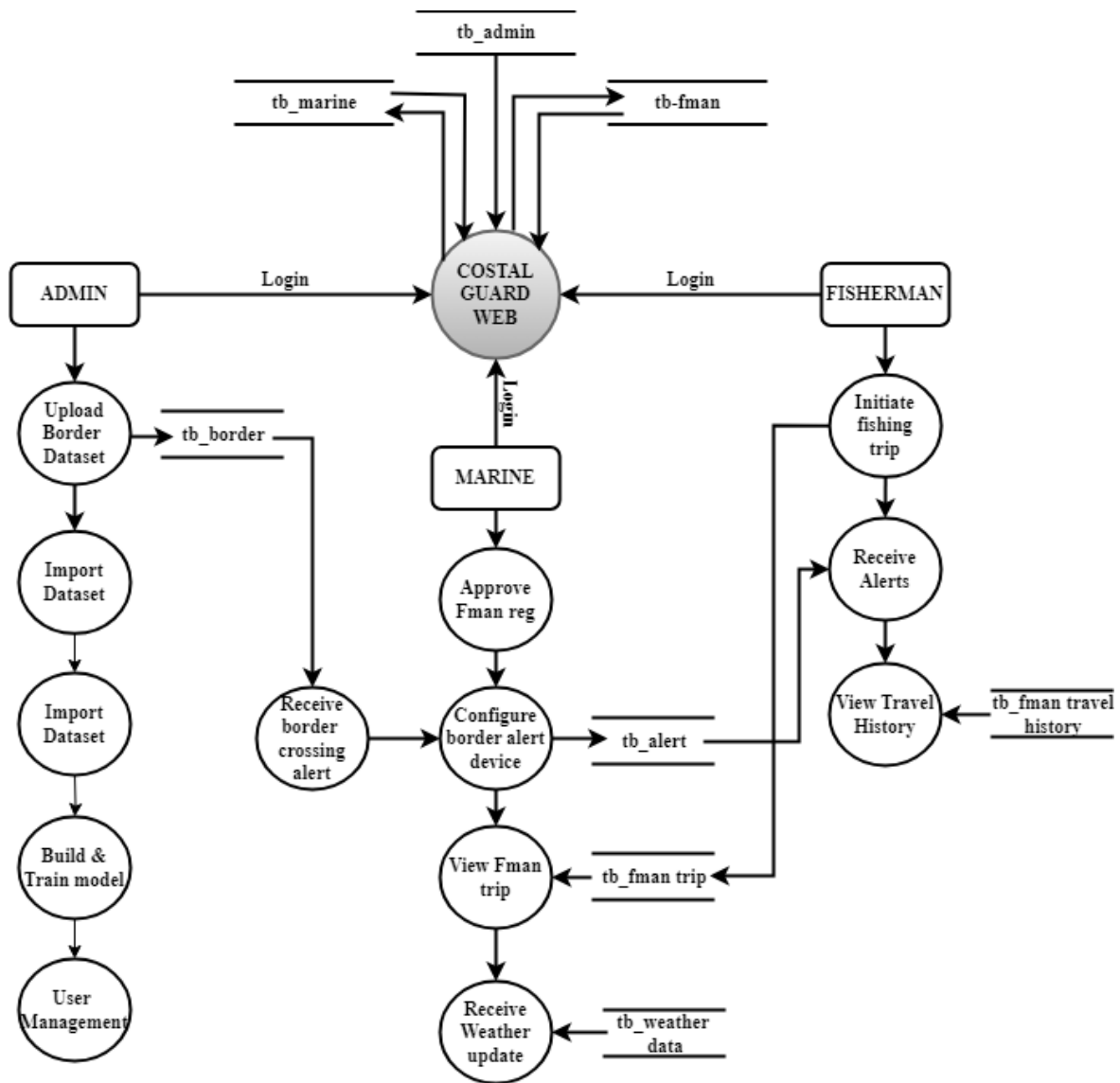


Fig. 6.2.3. Dataflow Diagram Level 2

6.3. UML DIAGRAMS

6.3.1. Use Case UML

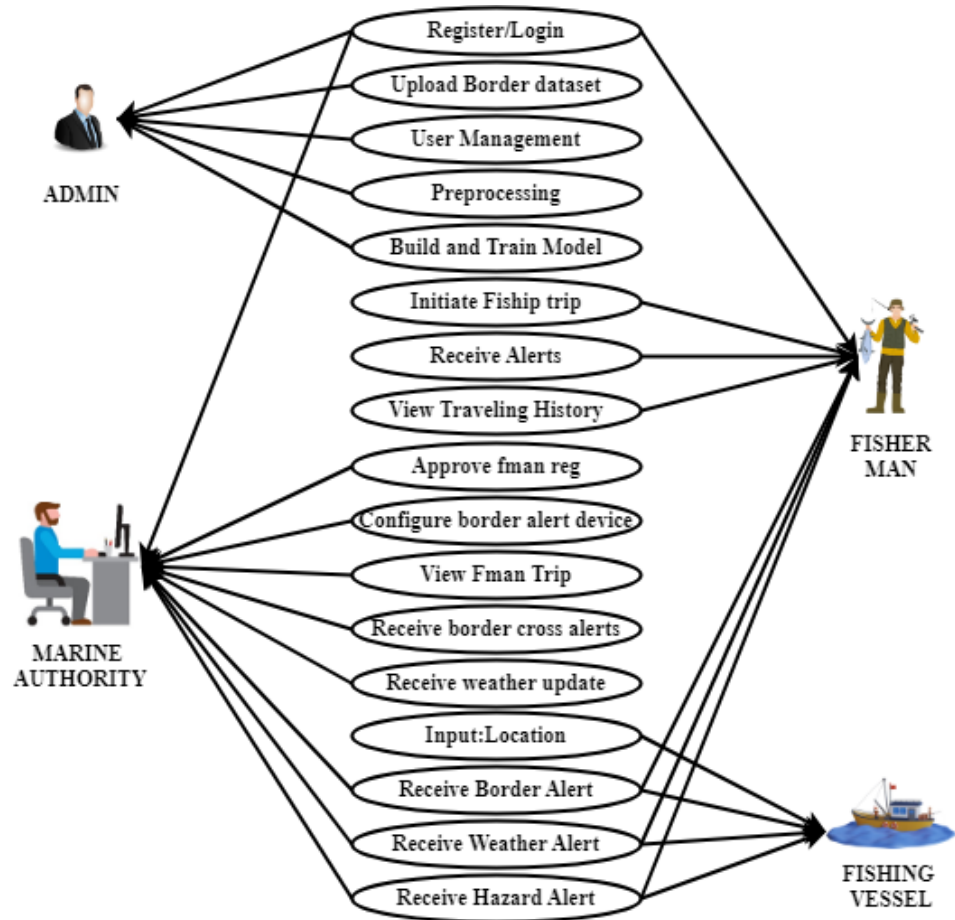


Fig. 6.3.1. Use Case Diagram

6.4. Database Design

Table 6.4.1. User Table

S.No	Field	Data Type	Field Size	Constraint	Description
1	user_id	INT	11	Primary Key, Auto Increment	Unique user ID
2	name	VARCHAR	100	NOT NULL	Name of the user
3	email	VARCHAR	100	UNIQUE, NOT NULL	Email address of user
4	phone	VARCHAR	15	NULL	Contact number
5	role	VARCHAR	50	NOT NULL	Role of user (Admin, Fisherman, Officer, etc.)
6	password	VARCHAR	255	NOT NULL	Encrypted password
7	registration_date	DATETIME	-	NOT NULL	Date of user registration

Table 6.4.2. Vessel Tracking Table

S.No	Field	Data Type	Field Size	Constraint	Description
1	vessel_id	INT	11	Primary Key, Auto Increment	Unique vessel ID
2	user_id	INT	11	Foreign Key	Linked fisherman/user
3	vessel_name	VARCHAR	100	NOT NULL	Name of the vessel
4	latitude	FLOAT	-	NOT NULL	Current latitude
5	longitude	FLOAT	-	NOT NULL	Current longitude
6	timestamp	DATETIME	-	NOT NULL	Last updated time
7	status	VARCHAR	50	DEFAULT 'Active'	Vessel activity status

Table 6.4.3. Weather Data Table

S.No	Field	Data Type	Field Size	Constraint	Description
1	weather_id	INT	11	Primary Key, Auto Increment	Unique weather record ID
2	location	VARCHAR	100	NOT NULL	Location name
3	temperature	FLOAT	-	NOT NULL	Temperature in Celsius
4	wind_speed	FLOAT	-	NOT NULL	Wind speed in km/h
5	humidity	FLOAT	-	NOT NULL	Humidity percentage
6	condition	VARCHAR	100	NOT NULL	Weather condition (Rainy, Cloudy, etc.)
7	timestamp	DATETIME	-	NOT NULL	Time of weather update

Table 6.4.4. Alert Table

S.No	Field	Data Type	Field Size	Constraint	Description
1	alert_id	INT	11	Primary Key, Auto Increment	Unique alert ID
2	user_id	INT	11	Foreign Key	Targeted user
3	alert_type	VARCHAR	100	NOT NULL	Type of alert (Border, Weather, Hazard)
4	message	TEXT	500	NOT NULL	Alert message details
5	priority	VARCHAR	20	NOT NULL	Priority level (High, Medium, Low)
6	timestamp	DATETIME	-	NOT NULL	Alert generated time
7	status	VARCHAR	20	DEFAULT 'Pending'	Status of alert (Sent, Acknowledged, etc.)

CHAPTER 7

SYSTEM IMPLEMENTATION

The implementation of the proposed system involves several key components and steps:

1. Backend Development

- Implementation of backend functionalities using Python and Flask framework.
- Setup and configuration of MySQL database for data storage and retrieval.
- Integration of machine learning capabilities using TensorFlow and Keras for predictive analytics.

2. Frontend Development

- Design and development of user interfaces using HTML, CSS, and JavaScript.
- Utilization of Bootstrap framework for responsive and mobile-friendly designs.
- Implementation of interactive features for data visualization and user interaction.

3. Database Management

- Creation of database schema to store user profiles, border surveillance data, alerts, and other relevant information.
- Implementation of database queries for CRUD operations (Create, Read, Update, Delete) to manage data effectively.
- Optimization of database performance through indexing, normalization, and query optimization techniques.

4. Deep Learning Model Development

- Collection and preprocessing of historical border surveillance data for training the BorderNet model.
- Design and implementation of LSTM neural network architecture for border breach prediction.
- Training and evaluation of the model using labeled datasets to optimize predictive accuracy.

5. Integration of External APIs

- Integration of weather data APIs to fetch real-time meteorological information for weather alerts and forecasts.
- Incorporation of GPS tracking APIs to receive continuous location updates from fishing vessels for border surveillance.

6. Alert Notification System:

- Development of alert notification mechanisms using SMS, email, and push notifications for timely dissemination of information.
- Configuration of notification triggers based on predefined thresholds and conditions for different types of alerts.

7. User Authentication and Access Control:

- Implementation of user authentication mechanisms to secure access to system functionalities.
- Assignment of user roles and permissions to control access to sensitive data and features.

8. Testing and Quality Assurance:

- Conducting unit tests, integration tests, and system tests to validate the functionality and reliability of the system.
- Identification and resolution of bugs, errors, and performance bottlenecks through iterative testing and debugging processes.

7.1. SYSTEM DESCRIPTION

The project is developed using Python, Flask, MySQL, and Wampserver for backend infrastructure and local server hosting. Machine learning capabilities, facilitated by TensorFlow and Keras, enable predictive analytics, while Pandas, NumPy, Matplotlib, and Seaborn handle data processing and visualization. Bootstrap ensures a responsive frontend for coast guard personnel, enhancing usability. This centralized platform facilitates real-time border surveillance, communication with authorities, and coordination with fishermen. Authentication ensures secure access, while features like BorderAlert device management and integration with the BorderNet model enable proactive monitoring. Immediate alerts for breaches, weather changes, and hazards, along with reporting tools and user management, bolster maritime security. Build and Train Module involves dataset description, importation, preprocessing, feature extraction, and classification. The dataset comprises various features like timestamp, latitude, longitude, and labels indicating border status (safe/breached). Data preprocessing addresses null values, redundant data, and outliers, ensuring quality input for the model. Feature extraction leverages correlation analysis to select relevant features for training. Classification employs LSTM neural networks to classify borders based on temporal dependencies in the data. Key components of the network architecture include multiple layers, hidden units, activation functions, dropout, and recurrent connections. Hyperparameter specification, model compilation, training, and validation refine the model's performance, culminating in deployment within the Coast Guard Stations Web App. The Border Crossing Prediction Module anticipates potential border crossings by fishing vessels through real-time location tracking and predictive analytics. A dynamic sailing animation interface visualizes vessel movements, aiding in anomaly detection. Integrated BorderAlert GPS devices continuously transmit vessel location data, enabling real-time tracking. Predictive analytics,

powered by the BorderNet Model, utilize LSTM neural networks to forecast potential border breaches by analyzing historical vessel trajectories, maritime boundaries, and border activity. This proactive approach enhances maritime security by enabling timely intervention. The Alert System Module provides timely alerts to fishermen and maritime authorities regarding potential risks and hazards. Border alerts notify both parties of detected breaches or suspicious activities, weather alerts warn fishermen of adverse conditions, and hazard alerts highlight potential dangers. These alerts are disseminated through various communication channels, enhancing situational awareness and safety at sea. The System User Module facilitates interaction among stakeholders, including maritime authorities, fishermen, and weather data providers. Maritime authorities can log in securely, approve fishermen's registration requests, configure BorderAlert devices, receive border crossing alerts, access weather information, and view fishermen travel details. Fishermen can register, update profiles, initiate fishing trips, receive alerts, and view travel history. The weather data provider API updates real-time weather information to the Coast Guard Stations Web App, aiding informed decision-making.

7.2. MODULES DESCRIPTION

1. Coast Guard Stations Web App

The Coast Guard Stations Web App is being developed using Python and Flask for the backend, MySQL for database management, and Wampserver for local server hosting. Integration of machine learning capabilities is achieved with TensorFlow and Keras for predictive analytics, supported by Pandas, Scikit Learn, NumPy, Matplotlib, and Seaborn for data processing, visualization, and model training. The frontend is designed with Bootstrap to ensure a responsive and user-friendly interface, enhancing usability for coast guard personnel managing border security operations. This module serves as a centralized platform for maritime

security operations, facilitating real-time border surveillance, communication with authorities, and coordination with fishermen. With features like authentication for secure access, management of BorderAlert devices, and integration with the predictive BorderNet model, it ensures proactive monitoring and response to threats. The app delivers immediate alerts for breaches, weather changes, and hazards, while providing reporting tools for trend analysis and user management for access control. Its customizable interface fosters usability, making it a vital tool for enhancing maritime security.

2. BorderNet Model: Build and Train

The BorderNet Model: Build and Train Module encompasses several key steps in developing a robust predictive model for classifying borders as either safe or breached. Here's a detailed description of each component:

- **2.1. Dataset Description**

The dataset for the Tamil Nadu to Sri Lanka border consists of various features and labels aimed at training the Bordernet model to classify borders as safe or breached. Here's an example of what the dataset might look like:

Features

- **Timestamp:** Date and time of the border observation.
- **Latitude:** Geographic latitude of the border location.
- **Longitude:** Geographic longitude of the border location.
- **Speed:** Speed of vessels in the vicinity of the border.
- **Direction:** Direction of vessel movement relative to the border.
- **Weather Conditions:** Meteorological data such as wind speed, visibility, and precipitation.
- **Temperature:** Ambient temperature at the border location.

Labels:

- **Border Status:** Classification of the border as either "safe" or "breached" based on observed activity and security assessments.

Example Entry

Table:

Timestamp	Lat	Lon	Spd	Dir	WC	Temp	SS	MT	BS
2023-05-15 10:00:00	9.850	79.976	12	NE	Clear	28°C	Moderate	Moderate	Safe
2023-05-15 11:00:00	9.845	79.972	8	N	Partly Cloudy	29°C	Moderate	Light	Breached
2023-05-15 12:00:00	9.840	79.968	10	NW	Clear	30°C	Moderate	Heavy	Safe

In this example, each row represents an observation of the border at a specific timestamp. Features such as latitude, longitude, speed, direction, weather conditions, temperature, sea state, and marine traffic are recorded. The "Border Status" column indicates whether the border was classified as "safe" or "breached" based on observed activity. This dataset serves as the foundation for training the BorderNet model, enabling it to learn patterns and relationships between features and border status labels. By analyzing historical data and identifying relevant patterns, the model can effectively classify future observations of the border, contributing to enhanced maritime security along the Tamil Nadu to Sri Lanka border.

2.2. Import Dataset

The Import Dataset module serves to gather, organize, and load the dataset required for training and testing the BorderNet model. Its primary function is to import data from various sources and format it into a structured format suitable for

model training. It is implemented using Python libraries such as Pandas, NumPy, and Matplotlib for data manipulation and visualization. It utilizes functions and classes to automate data loading and visualization tasks, streamlining the dataset preparation process.

2.3. Preprocessing

The Preprocessing Module aims to ensure that the dataset used for training the BorderNet model is clean, consistent, and suitable for analysis. It focuses on handling null values, redundant data, and missing values, as well as performing data cleaning and removing duplicate entries. This module is implemented using Python libraries such as Pandas and NumPy for data manipulation and preprocessing tasks. Custom functions and pipelines are used to automate preprocessing steps and ensure reproducibility across datasets.

Handling Null Values

The module identifies and handles null values within the dataset using appropriate techniques such as imputation or deletion. Imputation methods like mean, median, or mode replacement may be used to fill missing values in numerical features, while categorical features may be imputed with the most frequent value.

Handling Redundant Data

Redundant data, which includes duplicate observations or features with identical information, is identified and removed from the dataset. This ensures that each observation contributes unique information to the model training process, preventing bias and overfitting.

Output

The output of the Preprocessing Module is a cleaned and standardized dataset ready for model training. This dataset is free from null values, redundant data, and missing values, and has undergone data cleaning to address errors and inconsistencies. It serves as the input to the model training process, ensuring that the BorderNet model is trained on high-quality data.

2.4. Classification

The Classification Module aims to implement a classification algorithm, specifically LSTM neural networks, to classify borders into two categories: safe or breached. By leveraging the temporal dependencies inherent in border surveillance data, LSTM networks can effectively capture complex patterns and dynamics, making them well-suited for this task. It is implemented using Python libraries such as TensorFlow and Keras for building and training LSTM neural networks. Custom functions and classes are utilized to define the network architecture, preprocess data, train the model, and evaluate its performance.

Key Components:

Number of Layers

The LSTM network may consist of one or more layers of LSTM cells stacked on top of each other. Deeper networks with multiple layers can capture more complex patterns and relationships in the data but may require longer training times and are more prone to overfitting.

Hidden Units

Each LSTM cell in the network contains a set of hidden units, also known as neurons or memory cells. Larger numbers of hidden units enable the network to capture more intricate dependencies but may increase computational complexity.

Recurrent Connections

LSTM networks contain recurrent connections that allow information to flow across time steps, enabling the network to process sequential data and capture long-term dependencies. These connections facilitate the retention and utilization of historical information, enhancing the network's predictive capabilities.

2.5. Deploy Model

The Deploy Model Module aims to seamlessly integrate the trained LSTM model into the Coast Guard Stations Web App, allowing coast guard personnel to access its predictive capabilities for real-time border classification. This integration empowers personnel with actionable insights to enhance border security operations. This involves configuring the web app to load the model and utilize its predictive capabilities for border classification tasks.

3. Border Crossing Prediction

The Border Crossing Prediction Module serves as a proactive tool in maritime security efforts, anticipating potential border crossings by fishing vessels through a combination of real-time location tracking and predictive analytics. Here's a detailed description of each component within this module.

3.1. Fishing Vessels Sailing Animation:

This component provides a dynamic visualization of fishing vessels' movements through a sailing animation interface. It displays real-time updates of vessel positions and trajectories, allowing coast guard personnel to monitor maritime activity visually. The animation provides an intuitive understanding of vessel movements, facilitating rapid detection of suspicious behavior or deviations from expected routes.

3.2. Continuously Input Location

Integrated BorderAlert GPS devices are installed on fishing vessels to continuously transmit location data to the Border Crossing Prediction Module. These devices provide accurate and up-to-date information on vessel positions, enabling the module to track vessel movements in real time. By receiving continuous location inputs, the module ensures that coast guard authorities have immediate access to the latest data on vessel positions and trajectories.

3.3. Predict Border Crossing

The predictive capability of the module relies on the trained BorderNet Model, which utilizes LSTM neural networks to forecast potential border crossings by fishing vessels. By analyzing historical data on vessel trajectories, maritime boundaries, and border activity, the model identifies patterns and trends indicative of potential breaches. Leveraging sequential data and capturing temporal dependencies, the model accurately predicts the likelihood of fishing vessels crossing into restricted areas or breaching maritime borders.

4. Alert System

The Alert System Module is designed to provide timely and relevant alerts to fishermen and maritime authorities, helping them mitigate risks and ensure safety in maritime environments. It plays a crucial role in enhancing safety and security in maritime environments by providing timely alerts to fishermen and maritime authorities regarding potential risks and hazards.

4.1. Border Alert

This component issues alerts to both fishermen and maritime authorities in the event of detected border breaches or suspicious activity. Alerts may include notifications via mobile applications, SMS, or email, informing fishermen of potential security threats and prompting them to take appropriate action.

Simultaneously, maritime authorities receive alerts to coordinate response efforts, deploy resources, and investigate potential security breaches.

4.2. Weather Alert

The Weather Alert component notifies fishermen of adverse weather conditions that may pose risks to their safety or affect fishing operations. Alerts are issued based on real-time meteorological data, providing information on factors such as strong winds, heavy rainfall, thunderstorms, or rough seas. Fishermen receive weather alerts through various channels, including mobile apps, SMS, or radio broadcasts, enabling them to make informed decisions and take necessary precautions to ensure their safety at sea.

4.3. Hazard Alert

The Hazard Alert component warns fishermen about potential hazards or dangers in their vicinity, such as submerged rocks, navigational hazards, or other vessels in distress. Alerts are generated based on data from maritime surveillance systems, AIS (Automatic Identification System), or crowd-sourced reports from other vessels. Fishermen receive hazard alerts through accessible communication channels, enabling them to navigate safely and avoid potentially risky situations while at sea.

5. System User

The System User Module facilitates interaction between various stakeholders, including maritime authorities, fishermen, and weather data providers, within the maritime security system. Here's a detailed description of each user category and their respective functionalities:

5.1. Maritime Authorities

Login: Maritime authorities can access the system by logging in with their credentials, ensuring secure access to sensitive information and functionalities.

Approve Fisherman Registration Request: Authorities review and approve registration requests from fishermen, validating their identity and credentials before granting access to the system.

Configure BorderAlert Device to Fisherman Vessel: Authorities configure BorderAlert devices on fishermen vessels, ensuring seamless integration with the surveillance system for real-time border monitoring.

Receive Border Crossing Alert of Fisherman: Authorities receive instant alerts in case of detected border crossings or suspicious activities by fishermen, enabling prompt response and intervention.

5.2. Weather Data Provider API

Update Weather Information to the Coast Guard Stations Web App: The weather data provider API updates real-time weather information to the Coast Guard Stations Web App, ensuring accurate and up-to-date meteorological data for informed decision-making by authorities and fishermen.

CHAPTER 8

SYSTEM TESTING

System testing of the project involved validating the functionality, performance, security, and usability of the entire system as a whole. Here's an overview of the testing process:

1. Functional Testing

- Verified that all features and functionalities outlined in the system requirements worked correctly.
- Tested user interactions such as login, registration, initiating fishing trips, receiving alerts, and viewing travel history.
- Ensured that the BorderNet model accurately classified borders as safe or breached, and the alert system issued timely notifications.

2. Performance Testing

- Assessed the responsiveness and speed of the web application, especially during peak usage times.
- Tested the system's scalability to handle a large number of concurrent users and data processing tasks.
- Evaluated the response time of the predictive analytics module in forecasting border crossings and issuing alerts.

3. Security Testing

- Conducted penetration testing to identify and address vulnerabilities in the system, including SQL injection, cross-site scripting, and unauthorized access.
- Verified that user authentication mechanisms were robust and prevented unauthorized access to sensitive information.
- Ensured that data transmission and storage were encrypted to protect against data breaches and unauthorized access.

4. Usability Testing

- Gathered feedback from users to assess the ease of use, intuitiveness, and overall user experience of the system.
- Tested the accessibility of the system across different devices and screen sizes to ensure a consistent user interface.
- Identified any usability issues or pain points and made necessary improvements to enhance user satisfaction.

5. Compatibility Testing

- Tested the compatibility of the web application with various web browsers (e.g., Chrome, Firefox, Safari) and operating systems (e.g., Windows, macOS, Linux).
- Ensured that the system functioned correctly on different devices, including desktops, laptops, tablets, and smartphones.

6. Regression Testing

- Performed regression testing to ensure that recent updates or changes to the system had not introduced any new bugs or issues.
- Re-tested previously validated features and functionalities to confirm that they still worked as expected after modifications.

7. Integration Testing

- Verified the seamless integration between different modules and components of the system, including the web application, predictive analytics module, alert system, and user management module.
- Tested data flow and communication between modules to ensure proper functionality and data consistency.

8.1. TEST CASES

Test Cases for Maritime Authorities

1. Test Case ID: FBS_MA_TC_001

- **Input:** Maritime authority logs in with valid credentials.
- **Expected Result:** Authority should be successfully logged in and redirected to the dashboard.
- **Actual Result:** Authority was logged in successfully, and the dashboard was displayed.
- **Status:** Pass

2. Test Case ID: FBS_MA_TC_002

- **Input:** Maritime authority approves a fisherman registration request.
- **Expected Result:** Fisherman's registration status should be updated to approved, and a confirmation message should be sent.
- **Actual Result:** Fisherman's registration status was updated to approved, and a confirmation email was sent.
- **Status:** Pass

3. Test Case ID: FBS_MA_TC_003

- **Input:** Maritime authority configures BorderAlert device to a fisherman vessel.
- **Expected Result:** Device should be successfully configured and linked to the fisherman's vessel for border monitoring.
- **Actual Result:** BorderAlert device was successfully configured and linked to the fisherman's vessel.
- **Status:** Pass

4. Test Case ID: FBS_MA_TC_004

- **Input:** Maritime authority receives a border crossing alert of a fisherman.

- **Expected Result:** Authority should receive a real-time alert notification with details of the border breach.
- **Actual Result:** Authority received an alert notification "Border breached by fisherman XYZ at coordinates at timestamp."
- **Status:** Pass

Test Cases for Fishermen

1. Test Case ID: FBS_F_TC_001

- **Input:** Fisherman registers with Aadhar and Fisherman Card.
- **Expected Result:** Fisherman's registration should be successfully submitted for approval.
- **Actual Result:** Fisherman's registration was submitted successfully, awaiting approval.
- **Status:** Pass

2. Test Case ID: FBS_F_TC_002

- **Input:** Fisherman receives registration approval.
- **Expected Result:** Fisherman should receive confirmation of registration approval.
- **Actual Result:** Fisherman received a confirmation email "Your registration has been approved."
- **Status:** Pass

3. Test Case ID: FBS_F_TC_003

- **Input:** Fisherman logs in with valid credentials.
- **Expected Result:** Fisherman should be successfully logged in and redirected to the dashboard.
- **Actual Result:** Fisherman was logged in successfully, and the dashboard was displayed.
- **Status:** Pass

4. **Test Case ID:** FBS_F_TC_004

- **Input:** Fisherman updates profile information.
- **Expected Result:** Fisherman's profile should be successfully updated with the new information.
- **Actual Result:** Fisherman's profile was updated successfully with the new contact details.
- **Status:** Pass

8.2. TEST REPORT

Introduction

The project is undergoing testing to ensure its functionality, usability, and reliability in enhancing maritime security operations. This test report outlines the test objectives, scope, environment, results, and bug reports identified during testing.

Test Objective

The objective of this testing is to validate the functionality and performance of the Coast Guard Stations Web App, including features such as user authentication, border monitoring, alert systems, and integration with external data sources.

Test Environment

- **Backend Infrastructure:** Python, Flask, MySQL, Wampserver
- **Machine Learning Frameworks:** TensorFlow, Keras
- **Data Processing & Visualization:** Pandas, NumPy, Matplotlib, Seaborn
- **Frontend Framework:** Bootstrap
- **Testing Tools:** Selenium, JUnit
- **Operating System:** Windows 10
- **Browsers:** Chrome, Firefox, Edge

Test Result

The testing results indicate that the Coast Guard Stations Web App performed satisfactorily across various test scenarios, with the majority of test cases passing successfully. However, a few bugs were identified and reported during testing.

Bug Report

A bug report identifies any deviations from expected behavior observed during testing. Each bug is assigned a unique ID and linked to the corresponding test case where the issue occurred. The bug description provides details of the observed behavior, and the bug status indicates whether it has been resolved or remains open.

TABLE:

BID	TCID	Bug Description	Bug Status	Output
B001	FBS_MA_TC_004	Maritime authority does not receive alert	Fixed	Authority did not receive border alert email
B002	FBS_F_TC_006	Fisherman receives duplicate alert	Open	Fisherman received multiple border alerts

Test Conclusion

Overall, the project demonstrated robust functionality and usability, meeting the intended objectives of enhancing maritime security operations. The identified bugs have been documented and will be addressed in subsequent development iterations to ensure optimal performance and user experience.

CHAPTER 9

BOTTOM LINE

9.1. CONCLUSION

In conclusion, this project fortified with the BorderNet model and advanced alert systems, emerges as a robust solution for bolstering maritime security operations. Through the amalgamation of cutting-edge technologies like machine learning, real-time data integration, and predictive analytics, this project significantly elevates the efficacy of border surveillance and response mechanisms. The user-centric design of the system, coupled with its ability to customize alert preferences, ensures accessibility and usability for coast guard personnel and fishermen. By providing continuous monitoring of maritime borders and issuing timely alerts for potential breaches, adverse weather conditions, and maritime hazards, the system plays a pivotal role in enhancing safety and security at sea. Moreover, the feasibility analysis underscores the project's practicality and potential for real-world implementation. Through meticulous system implementation, including input and output design, the project demonstrates technical competence and attention to detail. This meticulous approach ensures that each component of the system functions seamlessly, contributing to its overall effectiveness. In essence, this project represents a significant leap forward in maritime security technology, promising to revolutionize border surveillance operations and safeguard the interests of fishermen and maritime authorities alike. With comprehensive testing, deployment, and ongoing maintenance, the system stands poised to fulfill its objectives of enhancing maritime security, mitigating risks, and fostering safer maritime environments for all stakeholders.

9.2. FUTURE ENHANCEMENT

Future enhancements for the Coast Guard Stations Web App and associated systems encompass a range of possibilities aimed at further refining its capabilities and expanding its utility. Here are some potential avenues for improvement:

Integration of Additional Data Sources: Incorporating data from satellite imagery, unmanned aerial vehicles (UAVs), and other sensor networks can provide more comprehensive situational awareness, enabling better-informed decision-making.

Geofencing Capabilities: Introducing geofencing capabilities to define virtual boundaries and trigger alerts when vessels enter or exit designated areas can enhance targeted surveillance and response efforts.

Mobile Application Development: Developing dedicated mobile applications for coast guard personnel and fishermen can extend the reach of the system, enabling real-time access to alerts, status updates, and reporting functionalities from anywhere.

Incident Response Automation: Implementing automated incident response workflows that leverage artificial intelligence (AI) and natural language processing (NLP) can streamline response efforts and ensure rapid coordination between stakeholders in emergency situations.

APPENDIX

10.1 SOURCE CODE

Packages

```
from flask import Flask, render_template, Response, redirect, request, session,
abort, url_for
import os
import base64
import cv2
import pandas as pd
import numpy as np
import imutils
from flask import send_file
from werkzeug.utils import secure_filename
import shutil
import json
import csv
import subprocess
import mysql.connector
import hashlib
from datetime import datetime
from datetime import date
import datetime
import time
from random import seed
from random import randint
from PIL import Image
import stepic
```

```
import urllib.parse
from urllib.request import urlopen
import webbrowser
```

Database Connection

```
mydb = mysql.connector.connect(
host="localhost",
user="root",
password="",
charset="utf8",
database="fisherman"
```

Login

```
def login_fm():
msg=""
if request.method=='POST':
uname=request.form['uname']
pwd=request.form['pass']
cursor = mydb.cursor()
cursor.execute('SELECT * FROM fm_fisher WHERE uname = %s AND pass =
%s', (uname, pwd))
account = cursor.fetchone()
if account:
session['username'] = uname
return redirect(url_for('fm_home'))
else:
msg = 'Incorrect username/password!'
```

Fisherman Registration

```
def register():
```

```

msg=""
fn=""
email=""
mess=""

act=request.args.get("act")
mycursor = mydb.cursor()
mycursor.execute("SELECT max(id)+1 FROM fm_fisher")
maxid = mycursor.fetchone()[0]
if maxid is None:
    maxid=1
now = datetime.datetime.now()
rdate=now.strftime("%d-%m-%Y")
if request.method=='POST':
    name=request.form['name']
    gender=request.form['gender']
    dob=request.form['dob']
    address=request.form['address']
    mobile=request.form['mobile']
    email=request.form['email']
    pass1=request.form['pass']
    file = request.files['file']
    file2 = request.files['file2']
    uname="F"+str(maxid)
    filename=file.filename
    aadhar="A"+str(maxid)+filename
    file.save(os.path.join("static/upload", aadhar))
    filename2=file2.filename

```

```

fcard="F"+str(maxid)+filename2
file2.save(os.path.join("static/upload", fcard))
mess="Dear "+name+", Fisherman ID:"+uname+", Password:"+pass1
sql = "INSERT INTO
fm_fisher(id,name,gender,dob,address,mobile,email,aadhar,fisher_card,uname,pass
,rdate,approve_st) VALUES (%s,%s,%s, %s, %s, %s, %s, %s, %s, %s, %s,
%s,%s)"
val =
(maxid,name,gender,dob,address,mobile,email,aadhar,fcard,uname,pass1,rdate,'0')
mycursor.execute(sql, val)
mydb.commit()
print(mycursor.rowcount, "Registered Success")
msg="success"

```

Get Weather Data

```

lat1=lat[0:8]
lon1=lon[0:8]
responsetext="https://weather.visualcrossing.com/VisualCrossingWebServices/rest/
services/timeline/"+loc+"/"+sdate+"/"+edate+"?unitGroup=metric&include=days&
key=8STCAMRSRTEZ77JA2XRP7FMNC&contentType=csv"
cdata=[]
data1 = pd.read_csv(responsetext, header=0)
for ss in data1.values:
dt=[]
with open("static/weather_data.csv",'a',newline=") as outfile:
writer = csv.writer(outfile, quoting=csv.QUOTE_NONNUMERIC)
writer.writerow(ss)
cdata.append(dt)

```

Training

```
def process1():
    uname=""
    msg=""
    path='dataset/'
    df=pd.read_csv('static/indian-ocean-data.csv')
    dat1=df.head(100)
    data1=[]
    for ss1 in dat1.values:
        data1.append(ss1)
    #LSTM
    def load_data(stock, seq_len):
        amount_of_features = len(stock.columns)
        data = stock.as_matrix() #pd.DataFrame(stock)
        sequence_length = seq_len + 1
        result = []
        for index in range(len(data) - sequence_length):
            result.append(data[index: index + sequence_length])

    result = np.array(result)
    row = round(0.9 * result.shape[0])
    train = result[:int(row), :]
    x_train = train[:, :-1]
    y_train = train[:, -1][:,-1]
    x_test = result[int(row):, :-1]
    y_test = result[int(row):, -1][:,-1]
```

```

x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1],
amount_of_features))
x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], amount_of_features))
return [x_train, y_train, x_test, y_test]
def train(train_iter, dev_iter, test_iter, model, args):
if args.cuda:
model.cuda()
if args.Adam is True:
print("Adam Training.....")
optimizer = torch.optim.Adam(model.parameters(), lr=args.lr,
weight_decay=args.init_weight_decay)
elif args.SGD is True:
print("SGD Training.....")
optimizer = torch.optim.SGD(model.parameters(), lr=args.lr,
weight_decay=args.init_weight_decay,
momentum=args.momentum_value)
elif args.Adadelta is True:
print("Adadelta Training.....")
optimizer = torch.optim.Adadelta(model.parameters(), lr=args.lr,
weight_decay=args.init_weight_decay)
steps = 0
model_count = 0
best_accuracy = Best_Result()
model.train()
for epoch in range(1, args.epochs+1):
steps = 0
print("\n## The {} Epoch, All {} Epochs ! ##".format(epoch, args.epochs))

```



```

for batch in train_iter:
    feature, target = batch.text, batch.label.data.sub_(1)
    if args.cuda is True:
        feature, target = feature.cuda(), target.cuda()
    target = autograd.Variable(target) # question 1
    optimizer.zero_grad()
    logit = model(feature)
    loss = F.cross_entropy(logit, target)
    loss.backward()
    if args.init_clip_max_norm is not None:
        utils.clip_grad_norm_(model.parameters(), max_norm=args.init_clip_max_norm)
    optimizer.step()
    steps += 1
    if steps % args.log_interval == 0:
        train_size = len(train_iter.dataset)
        corrects = (torch.max(logit, 1)[1].view(target.size()).data == target.data).sum()
        accuracy = float(corrects)/batch.batch_size * 100.0
        sys.stdout.write(
            "\rBatch[{} / {}] - loss: {:.6f}  acc: {:.4f}%({} / {})".format(steps,
            train_size,
            loss.item(),
            accuracy,
            corrects,
            batch.batch_size))
    if steps % args.test_interval == 0:
        print("\nDev Accuracy: ", end="")
        eval(dev_iter, model, args, best_accuracy, epoch, test=False)

```

```

print("Test Accuracy: ", end="")
eval(test_iter, model, args, best_accuracy, epoch, test=True)
if steps % args.save_interval == 0:
    if not os.path.isdir(args.save_dir): os.makedirs(args.save_dir)
    save_prefix = os.path.join(args.save_dir, 'snapshot')
    save_path = '{}_steps{}.pt'.format(save_prefix, steps)
    torch.save(model.state_dict(), save_path)
    if os.path.isfile(save_path) and args.rm_model is True:
        os.remove(save_path)
    model_count += 1
    return model_count
def eval(data_iter, model, args, best_accuracy, epoch, test=False):
    model.eval()
    corrects, avg_loss = 0, 0
    for batch in data_iter:
        feature, target = batch.text, batch.label
        target.data.sub_(1)
        if args.cuda is True:
            feature, target = feature.cuda(), target.cuda()
        logit = model(feature)
        loss = F.cross_entropy(logit, target)
        avg_loss += loss.item()
        corrects += (torch.max(logit, 1)[1].view(target.size()).data == target.data).sum()
    size = len(data_iter.dataset)
    avg_loss = loss.item()/size
    accuracy = float(corrects)/size * 100.0
    model.train()

```

```

print(' Evaluation - loss: {:.6f}  acc: {:.4f}%({}/{}) '.format(avg_loss, accuracy,
corrects, size))
if test is False:
if accuracy >= best_accuracy.best_dev_accuracy:
best_accuracy.best_dev_accuracy = accuracy
best_accuracy.best_epoch = epoch
best_accuracy.best_test = True
if test is True and best_accuracy.best_test is True:
best_accuracy.accuracy = accuracy
if test is True:
print("The Current Best Dev Accuracy: {:.4f}, and Test Accuracy is :{:.4f}, locate
on {} epoch.\n".format(
best_accuracy.best_dev_accuracy, best_accuracy.accuracy,
best_accuracy.best_epoch))
if test is True:
best_accuracy.best_test = False
def build_model(layers):
model = Sequential()
model.add(LSTM(
input_dim=layers[0],
output_dim=layers[1],
return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(
layers[2],
return_sequences=False))
model.add(Dropout(0.2))

```

```

model.add(Dense(
output_dim=layers[2]))
model.add(Activation("linear"))
start = time.time()
model.compile(loss="mse", optimizer="rmsprop",metrics=['accuracy'])
print("Compilation Time : ", time.time() - start)
return model

def build_model2(layers):
d = 0.2
model = Sequential()
model.add(LSTM(128, input_shape=(layers[1], layers[0]),
return_sequences=True))
model.add(Dropout(d))
model.add(LSTM(64, input_shape=(layers[1], layers[0]),
return_sequences=False))
model.add(Dropout(d))
model.add(Dense(16,init='uniform',activation='relu'))
model.add(Dense(1,init='uniform',activation='linear'))
model.compile(loss='mse',optimizer='adam',metrics=['accuracy'])
return model

```

Fisherman Approval

```

def auth_home():
msg=""
uname=""
data4=[]
act=request.args.get("act")
#if 'username' in session:

```

```

#  uname = session['username']
ff=open("static/auth.txt","r")
uname=ff.read()
ff.close()
mycursor = mydb.cursor()
mycursor.execute("SELECT * FROM fm_authority where uname=%s",(uname,))
data = mycursor.fetchone()
mycursor.execute("SELECT * FROM fm_fisher")
data1 = mycursor.fetchall()
if act=="ok":
fid=request.args.get("fid")
mycursor.execute("update fm_fisher set approve_st=1 where id=%s",(fid,))
mydb.commit()
msg="yes"
data11 = pd.read_csv("static/weather_data.csv", header=0)
r=0
for ss in data11.values:
dt=[]

if ss[13]=="rain":
if float(ss[12])>0 and float(ss[11])>0:
r+=1

Fisherman Trip Request
def fm_trip():
msg=""
uname=""
data1=[]

```

```

act=request.args.get("act")
mycursor = mydb.cursor()
mycursor.execute("SELECT * FROM fm_fisher where uname=%s",(uname,))
rs = mycursor.fetchone()
fid=str(rs[0])
if request.method=='POST':
    sdate=request.form['sdate']
    sh=request.form['sh']
    sm=request.form['sm']
    sd=sdate.split("-")
    sdate1=sd[2]+"-"+sd[1]+"-"+sd[0]
    stime=sh+":."+sm
    mycursor.execute("SELECT max(id)+1 FROM fm_trip")
    maxid = mycursor.fetchone()[0]
    if maxid is None:
        maxid=1
    sql = "INSERT INTO
fm_trip(id,fisher_id,sdate,stime,edate,etime,request_st,trip_st) VALUES (%s, %s,
%s, %s, %s, %s, %s, %s)"
    val = (maxid,uname,sdate1,stime,"",'0','0')
    mycursor.execute(sql, val)
    mydb.commit()

```

10.2 SCREENSHOT

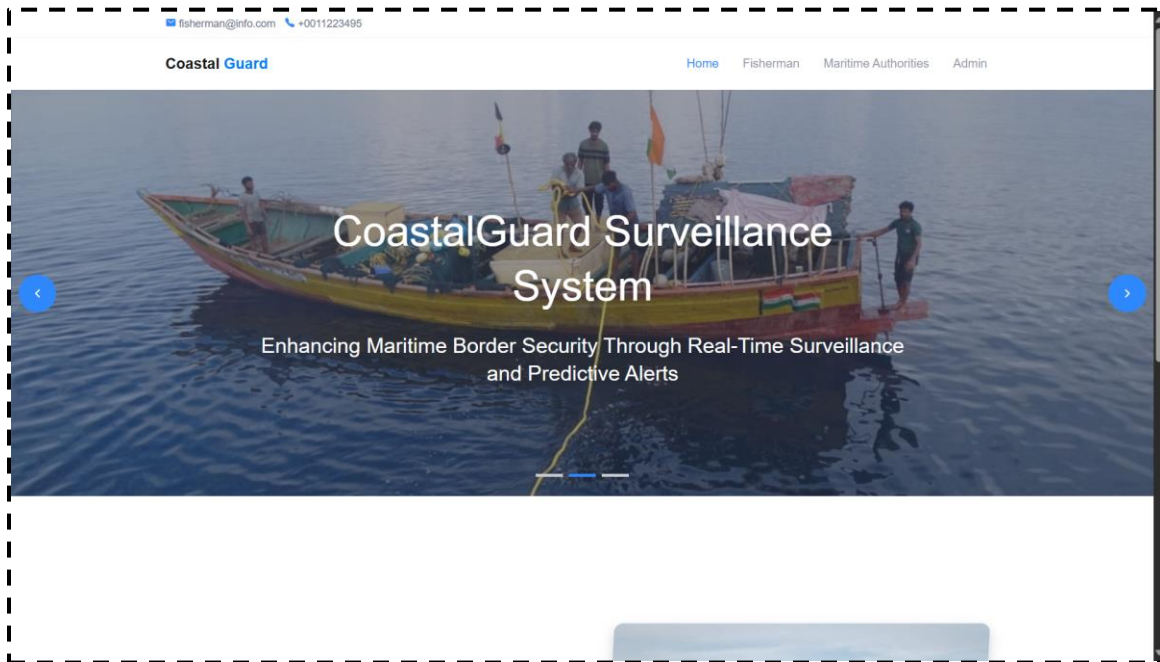


Fig:10.2.1 Screenshot 1

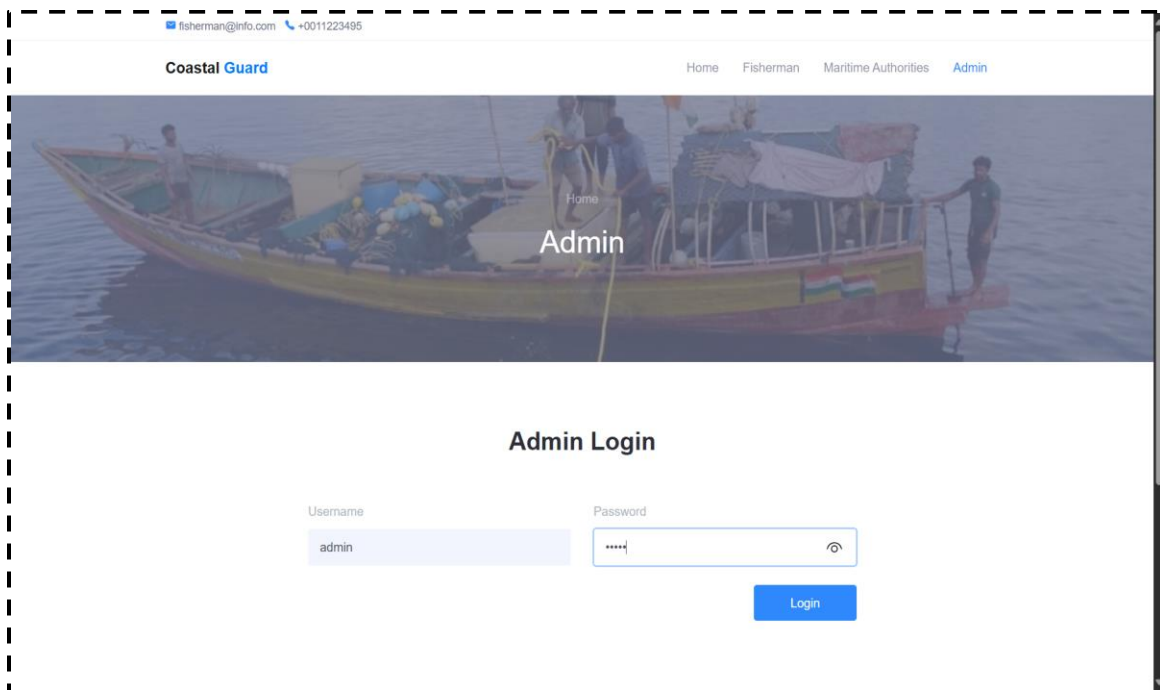


Fig:10.2.2 Screenshot 2

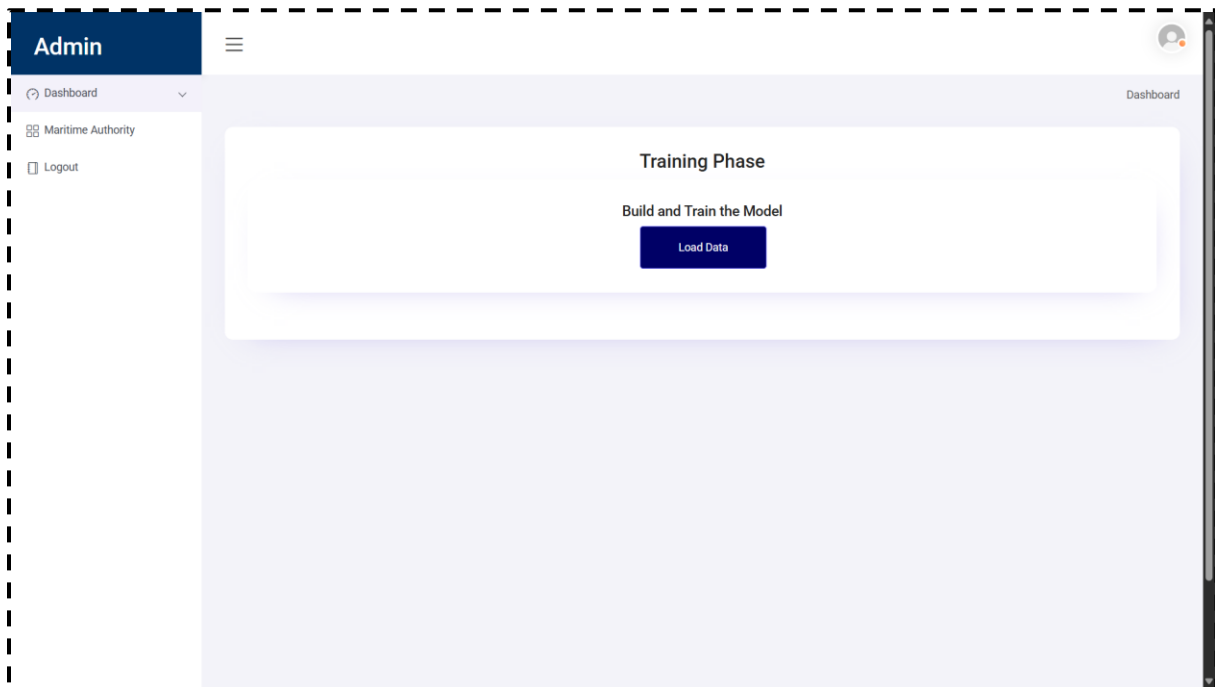


Fig:10.2.3 Screenshot 3

The screenshot shows the same Admin Dashboard, but the main content area displays a table titled 'Indian Ocean Location Data'. The table has three columns: '#', 'Latitude', and 'Longitude'. It contains 8 rows of data.

#	Latitude	Longitude
1	12.73835278	80.4748059
2	14.02089973	81.00214965
3	14.57449106	82.65009887
4	16.45868502	84.82539184
5	16.62719002	87.19843871
6	16.54295592	89.6154309
7	16.29003349	91.04365356
8	14.51068502	90.34052856

Fig:10.2.4 Screenshot 4

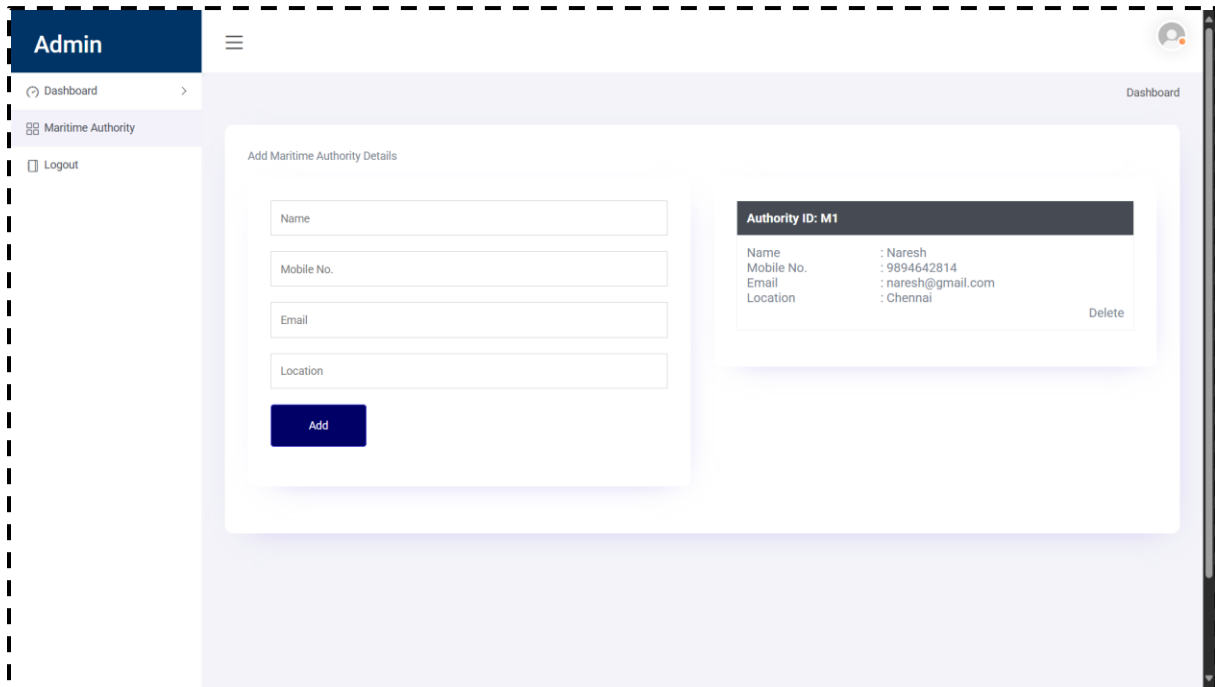


Fig:10.2.5 Screenshot 5

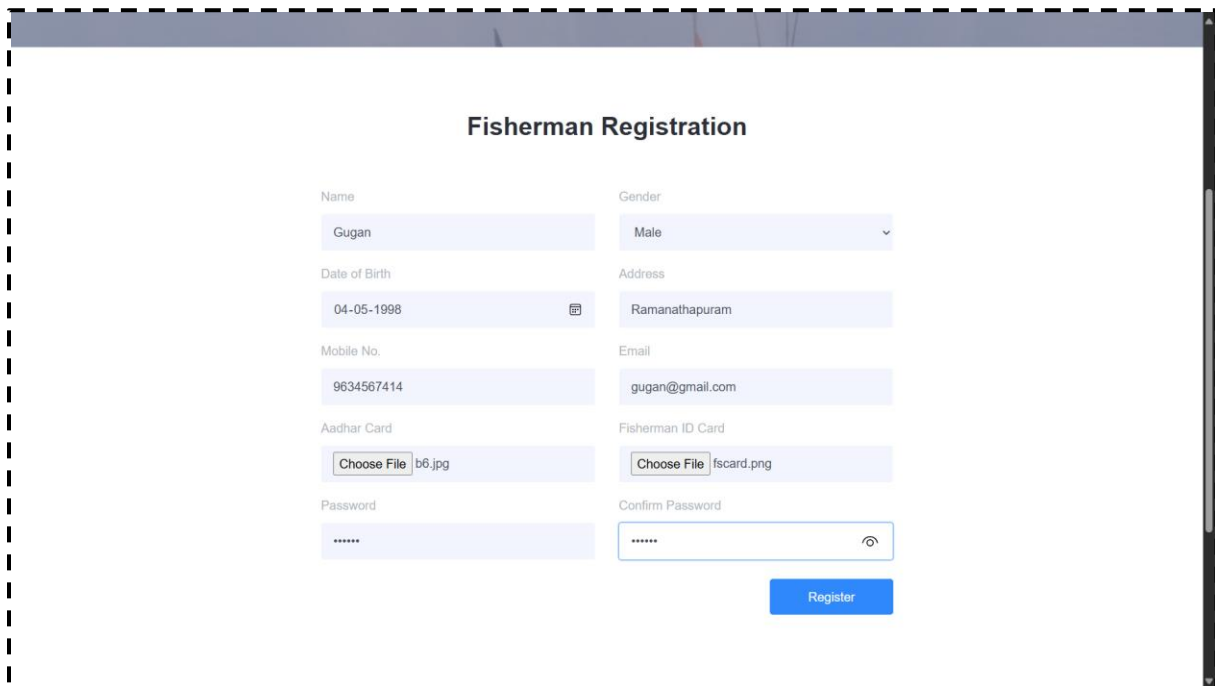


Fig:10.2.6 Screenshot 6

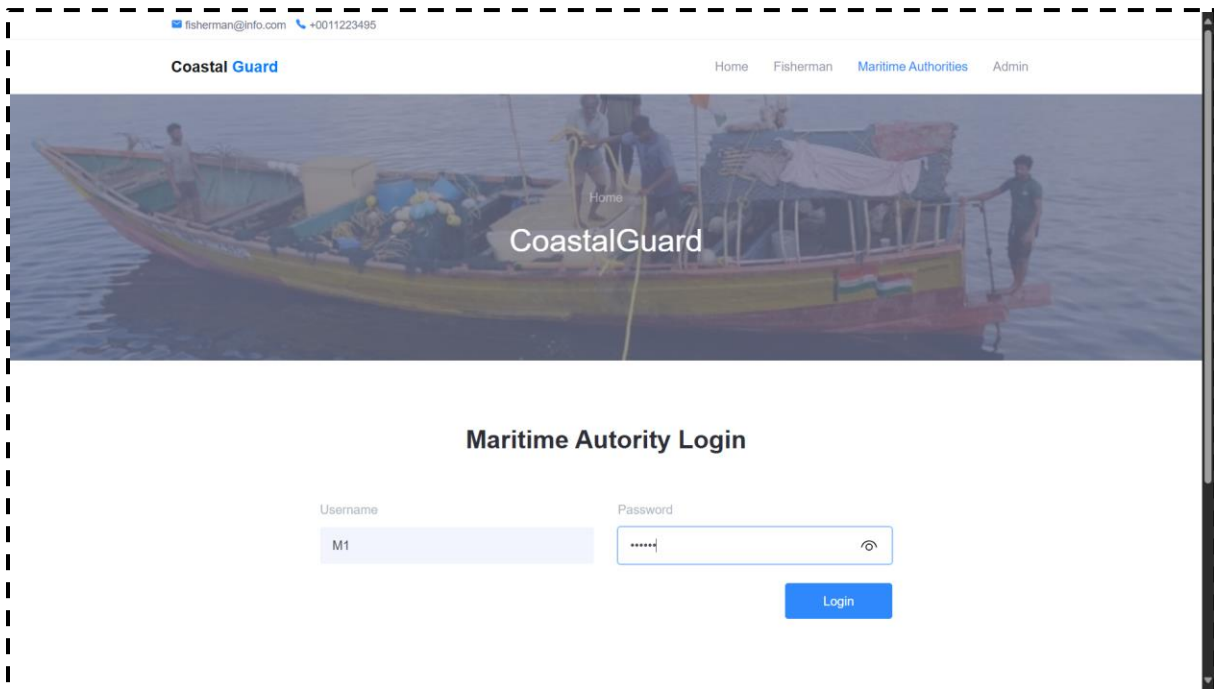


Fig:10.2.7 Screenshot 7

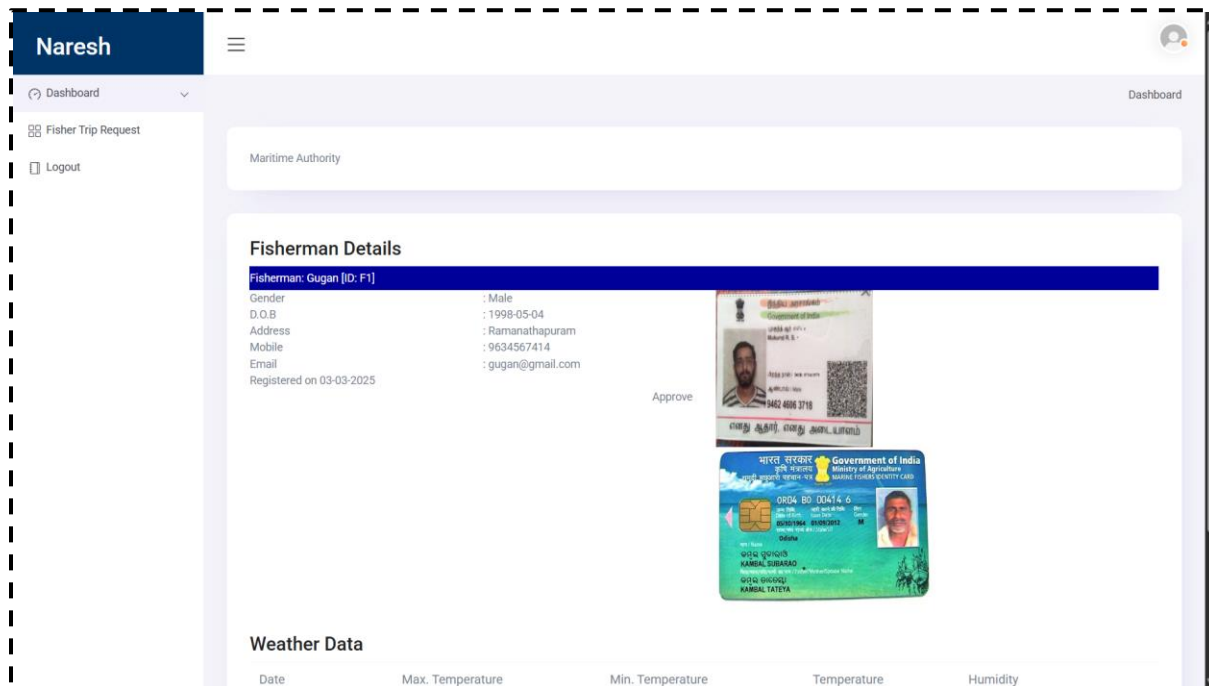


Fig:10.2.8 Screenshot 8

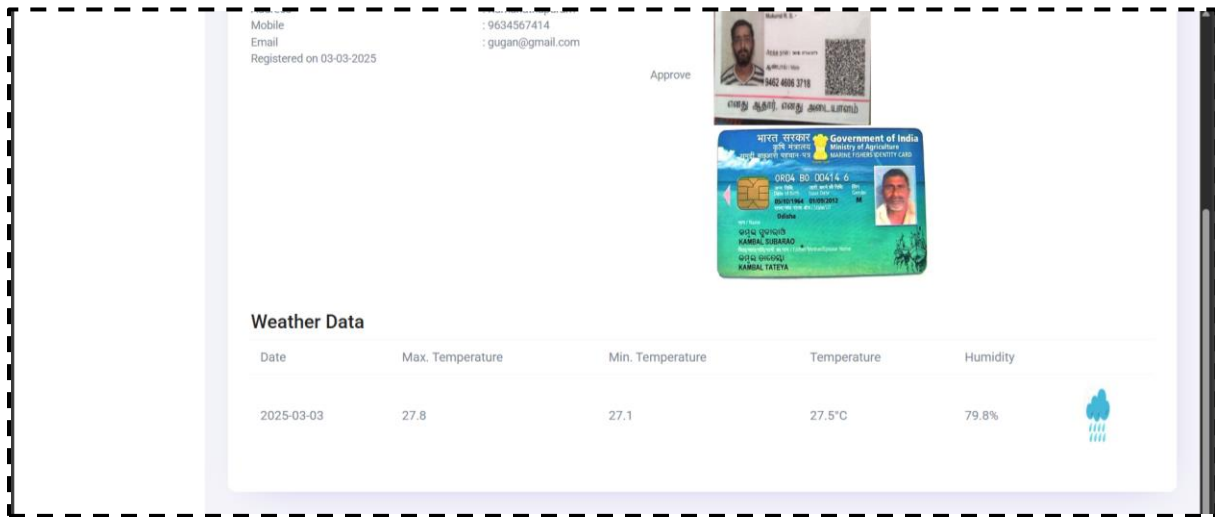


Fig:10.2.9 Screenshot 9

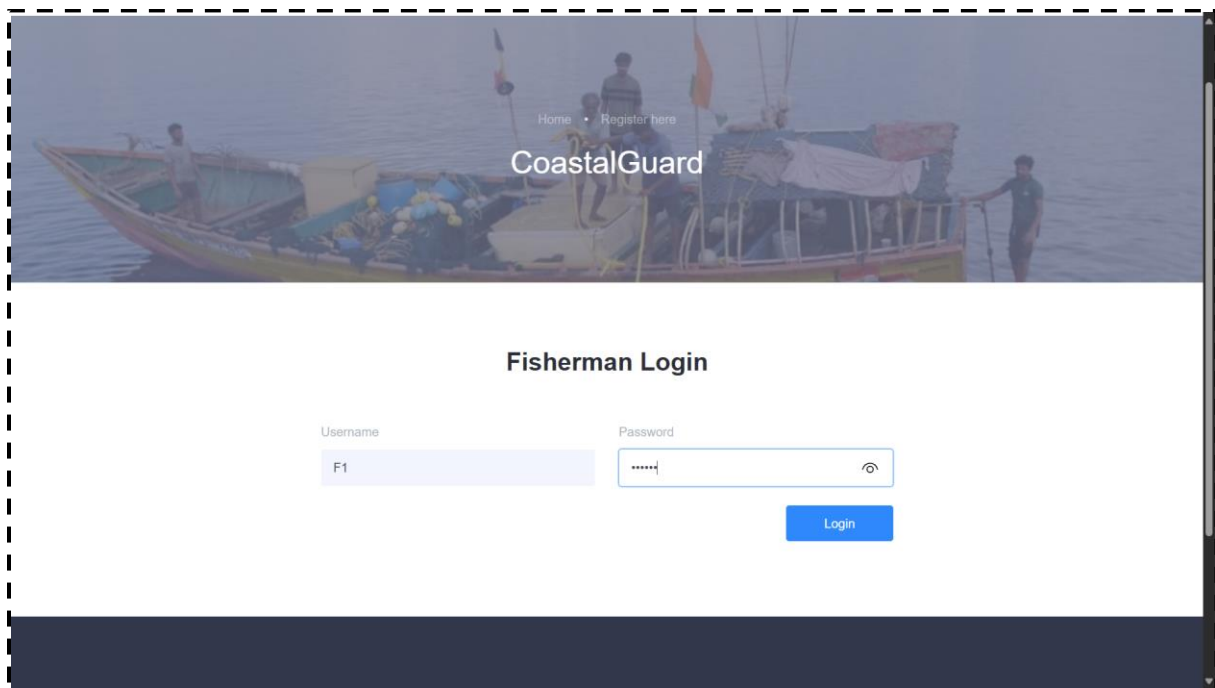


Fig:10.2.10 Screenshot 10

Fisher

Dashboard

Fishing Trip

Logout

Dashboard

Trip Request

Trip on

03-03-2025

16

10

Send

Fig:10.2.11 Screenshot 11

Fisher

Dashboard

Fishing Trip

Logout

Dashboard

Trip Request

Trip on

dd-mm-yyyy

-Hour-

-Minute-

Send

#	Trip on	End Trip	Request Status	Trip Status
1	03-03-2025 16:10		Request Sent	Not Started

Fig:10.2.12 Screenshot 12

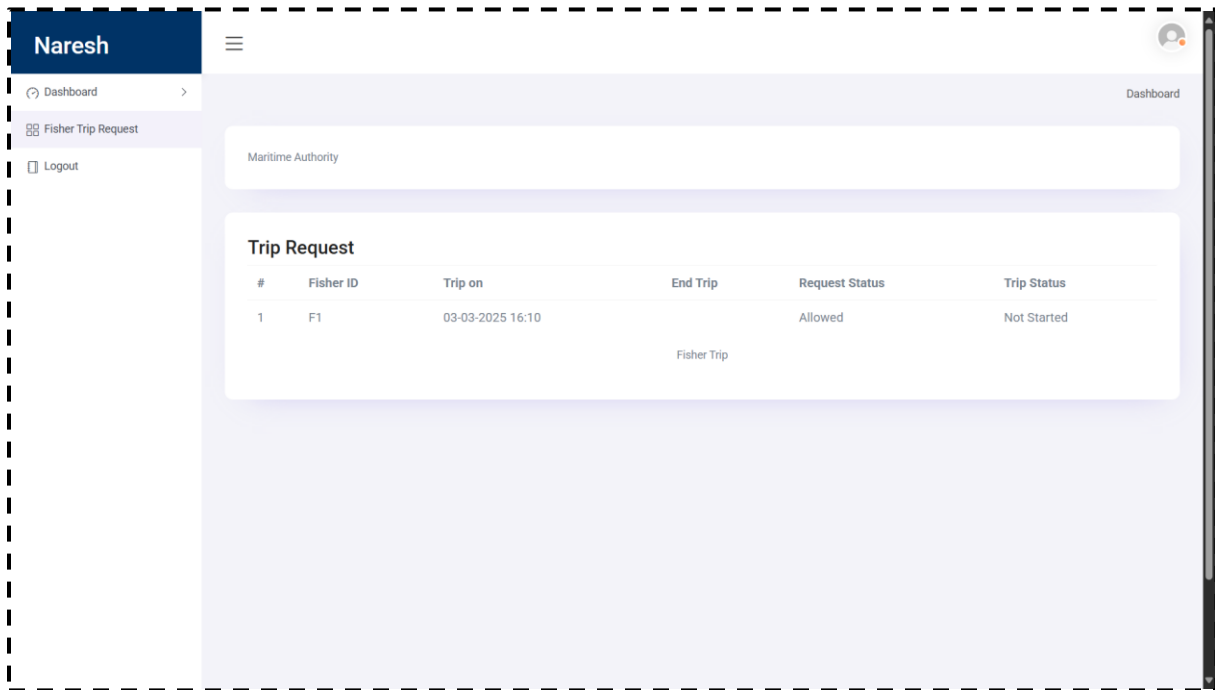


Fig:10.2.13 Screenshot 13

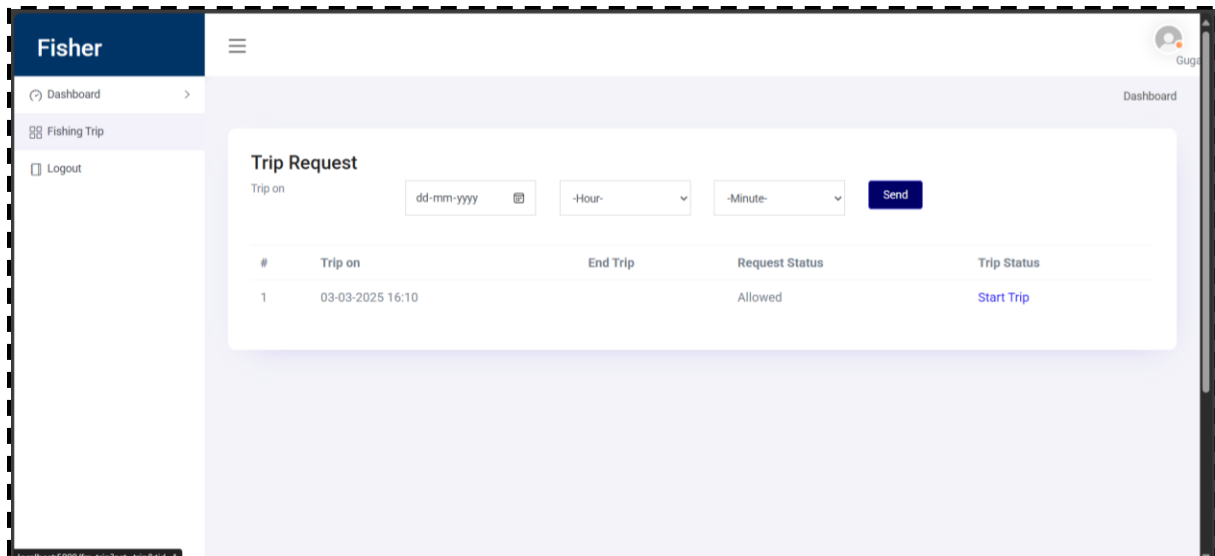


Fig:10.2.14 Screenshot 14

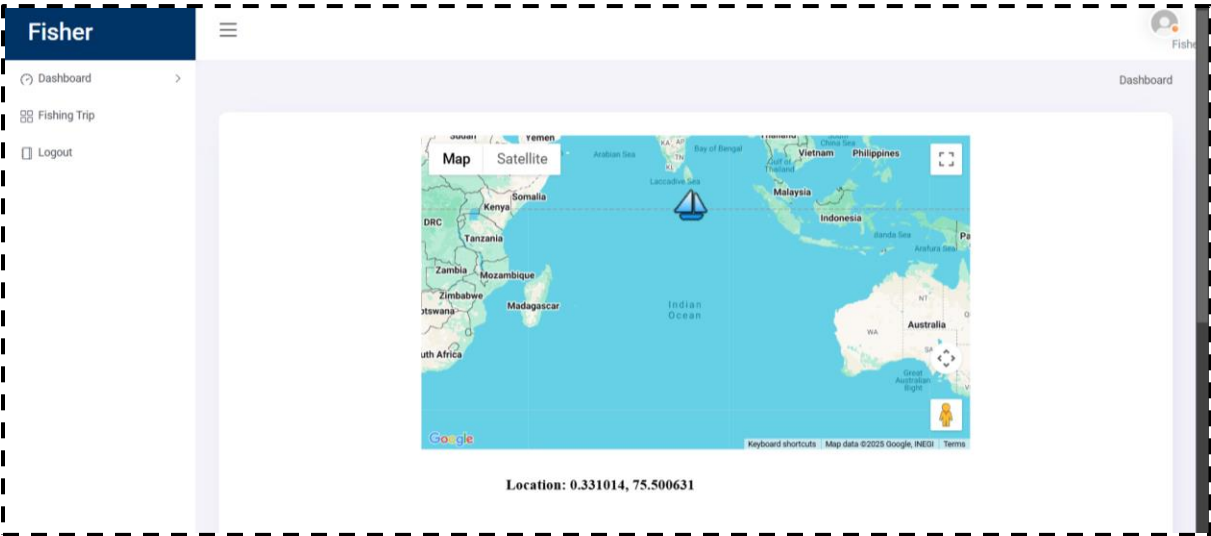


Fig:10.2.15 Screenshot 15

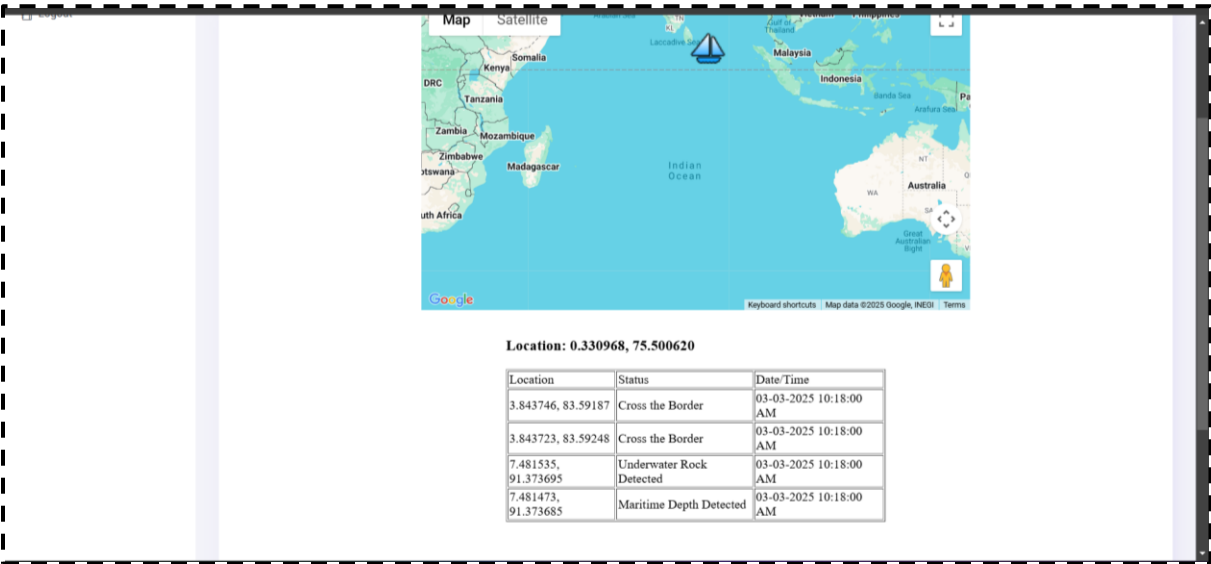


Fig:10.2.16 Screenshot 16

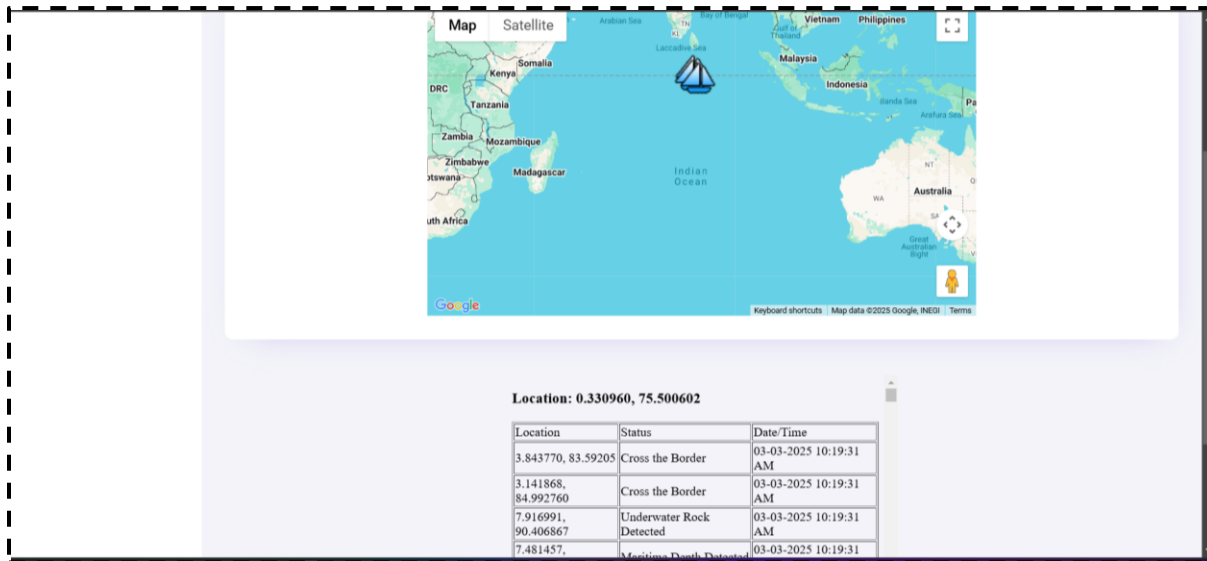


Fig:10.2.17 Screenshot 17

10.3 REFERENCES

JOURNAL REFERENCES

1. Muhammad Kahar and Mani Broto, "Management for Alleviating Poverty Among Fishermen Through Empowering Female Fishermen in North Kalimantan", *KnE Social Sciences*, 2023.
2. BA. Anandh, R Sakthivel et al., "Alert system of fisherman for secure navigation", *Journal of Physics: Conference Series*, 2022.
3. R. Tharwin Kumar, *Article Title: Fisherman Border Alert System Based on IOT Fisherman Border Alert System Based on IOT*, vol. 1, pp. 37-44, 2022.
4. Tk Kesavan and K Lakshmi, *GSM-Based Smart Marine Tracking System for Location Monitoring and Emergency Protection*, 2022.
5. Durka Devi Jayaram, Saravana Kumar Jayaprabha and John Clement Sunder, "FISHERMEN BORDER SECURITY ALERT SYSTEM USING IOT", *International Research Journal of Modernization in Engineering Technology and Science*, 2021.

6. S. V. S. N. Murthy, B. V. V. Satyanarayana and C. V. V. S. Srinivas, "Location Tracking and Warning System of a Ship using Arduino", *2021 5th International Conference on Computing Methodologies and Communication (ICCMC)*, pp. 1786-1790, 2021.
7. T. Jayaram, D. Durkka Devi and S. Jayaprabha, "Fishermen Border Security Alert System Using IOT", *International Research Journal of Modernization in Engineering Technology and Science*, vol. 3, no. 04, Apr. 2021.
8. S Armatha, D Roopasri, S Vinitha and S Vijayakumar, "Border Alert System and Emergency Contact for Fisherman using RSSI", *International Research Journal of Engineering and Technology (IRJET)*, 2020.
9. X. Liu, Q. Ruan, F. Miao and Z. Zhang, "Design of Early Warning System for Fishing Vessels in Coastal Area", *2020 3rd IEEE International Conference on Knowledge Innovation and Invention (ICKII)*, pp. 1-4, 2020.
10. A. S. M. N. Jahan, I. I. Uddin, M. N. M. Rone and N. Ahmed, "A Low-cost Boat Distress Alert and Safety System", *2020 IEEE Region 10 Symposium (TENSYP)*, pp. 1325-1328, 2020.
11. Kumar Harsh, Sarvesh Ranjan, Rishu Anand and Abhishek Kumar, "Border Detection System for Fisherman using GPS and", *Arduino International Journal of Engineering Research & Technology (IJERT)*, 2019.
12. K. T. Krishnamurthy, "A Low Cost Embedded Sea Navigation and Security Systems for Fisher Man", *2018 International Conference on Networking Embedded and Wireless Systems (ICNEWS)*, pp. 1-3, 2018.
13. T. Adiono, F. Dawani, E. Adinugraha, A. Rifai, M. Arijal, S. Fuada, et al., "Development of long-range communication system for fishermen: an initial study", *Proceedings of the IEEE international conference on ICT for Rural Development*, pp. 12-17, 2018.

- 14.N.K.K. Reddy, G. Ramakrishnan and K. Rajeshwari, "Ensuring fishermen safety through a range-based system by trizonal localization using low power RSSI", *Proceedings of the 4th IEEE international conference on Signal Processing Communication and Networking*, pp. 1-4, 2017.
- 15.R.D. Kumar, M.S. Aldo and J.C.F. Joseph, "Alert system for fishermen crossing border using Android", *Proceedings of the IEEE international conference on Electrical Electronics and Optimization Techniques*, pp. 4791-4795, 2016.

BOOK REFERENCES

1. "Python Crash Course" by Eric Matthes (Python Crash Course)
2. "Learning MySQL: Get a Handle on Your Data" by Russell J.T. Dyer (Learning MySQL)
3. "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron (Hands-On Machine Learning)
4. "Python for Data Analysis" by Wes McKinney (Python for Data Analysis)
5. "WampServer 2: Manually Installing the Apache, MySQL, and PHP" by Dr. James R. Small (WampServer 2)
6. "Bootstrap 4 Quick Start: Responsive Web Design and Development with Bootstrap 4" by Jacob Lett (Bootstrap 4 Quick Start)
7. "Fluent Python: Clear, Concise, and Effective Programming" by Luciano Ramalho (Fluent Python)
8. "High-Performance MySQL: Optimization, Backups, and Replication" by Baron Schwartz, Peter Zaitsev, Vadim Tkachenko (High-Performance MySQL)
9. "Python for Data Science For Dummies" by John Paul Mueller (Python for Data Science For Dummies)

WEB REFERENCES

1. Flask Documentation: Official documentation for Flask web framework - <https://flask.palletsprojects.com/en/2.0.x/>
2. MySQL Documentation: Official documentation for MySQL database management system - <https://dev.mysql.com/doc/>
3. TensorFlow Documentation: Official documentation for TensorFlow deep learning framework - <https://www.tensorflow.org/guide>
4. Pandas Documentation: Official documentation for Pandas data analysis library - <https://pandas.pydata.org/docs/>
5. Scikit-learn Documentation: Official documentation for Scikit-learn machine learning library - <https://scikit-learn.org/stable/documentation.html>
6. Matplotlib Documentation: Official documentation for Matplotlib data visualization library - <https://matplotlib.org/stable/contents.html>
7. NumPy Documentation: Official documentation for NumPy numerical computing library - <https://numpy.org/doc/stable/>
8. Seaborn Documentation: Official documentation for Seaborn statistical data visualization library - <https://seaborn.pydata.org/tutorial.html>
9. Bootstrap Documentation: Official documentation for Bootstrap CSS framework - <https://getbootstrap.com/docs/5.1/getting-started/introduction/>